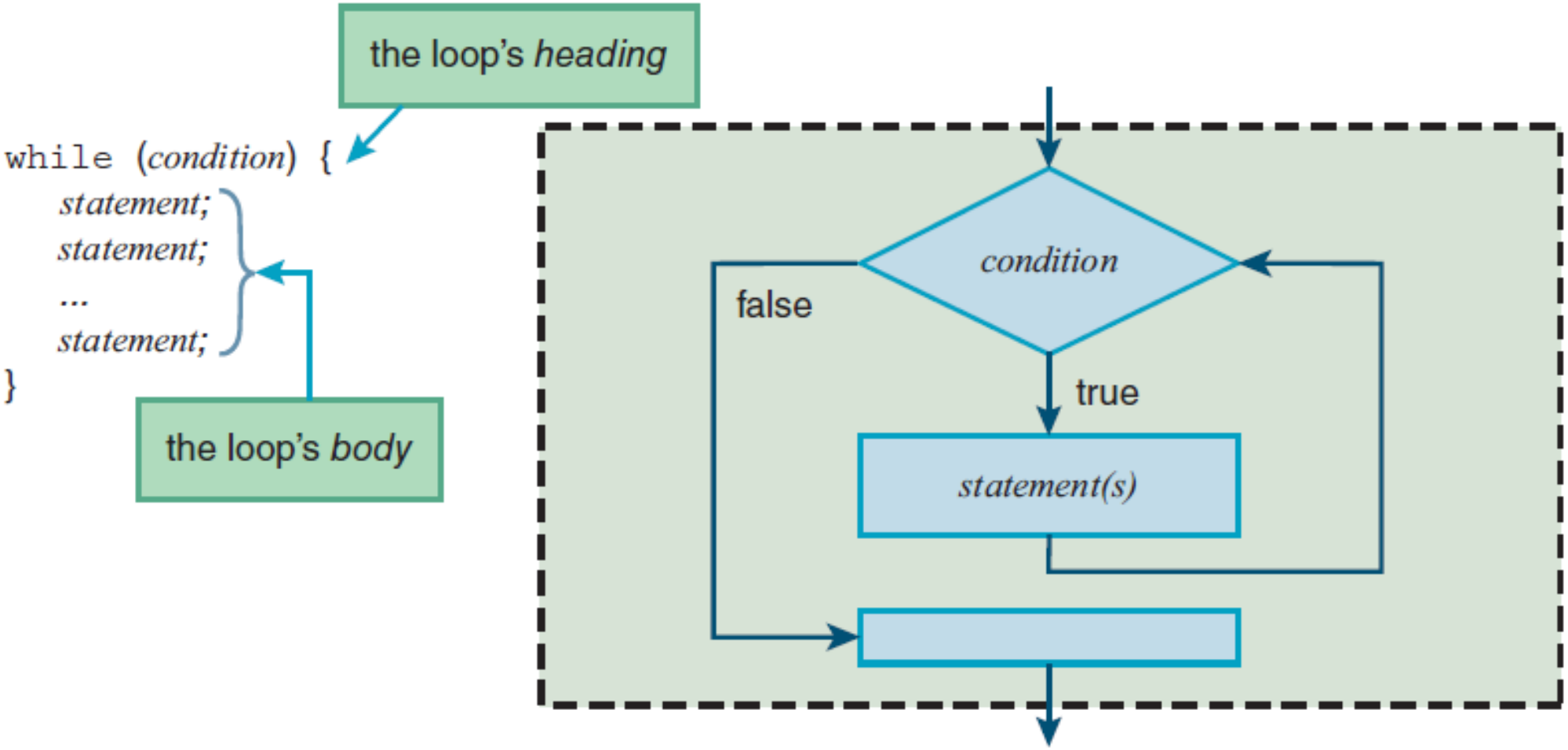
Module 5

Advanced JavaScript

Contents

While Loop, External JavaScript Files, do Loop, Radio Buttons, Checkboxes, for Loop - fieldset and legend Elements- Manipulating CSS with JavaScript- Using z-index to Stack Elements-Textarea Controls - Pull-Down Menus- List Boxes- Canvas and Drawing – Event Handler and Listener.

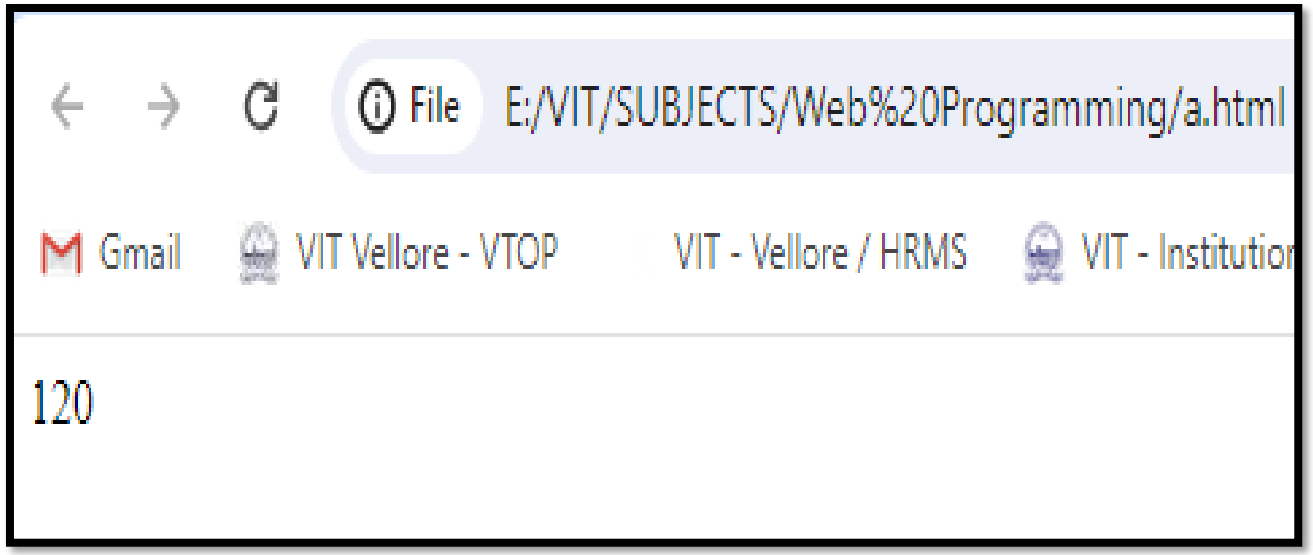
While Loop



Syntax and semantics for the `while` loop

while Loop

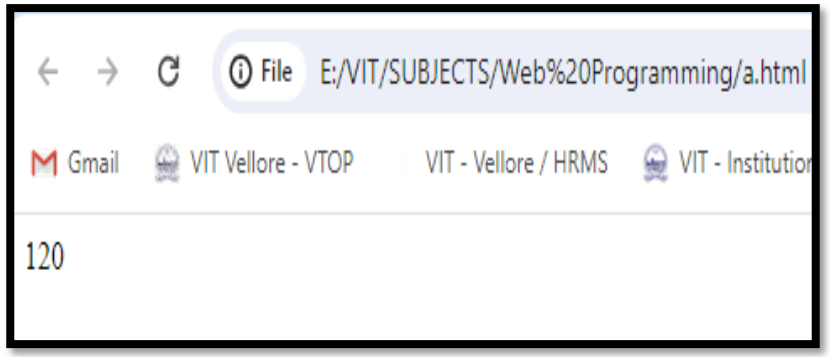
```
<html><body>
<script>
function factorial(n) {
    var ans = 1,i=1;
      if(n === 0)
        return 1;
    while(i<=n)
            {
        ans = ans * i;
         i++;
            }
    return ans;
}
 document.write(factorial(5));
</script>
</body></html>
```



External JavaScript Files

Fact.html

```
<html>
<head>
<title> arrays</title>
<meta charset="UTF-8">
<script type="text/javascript" src="a.js">
</script>
<body>
</body>
</html>
```

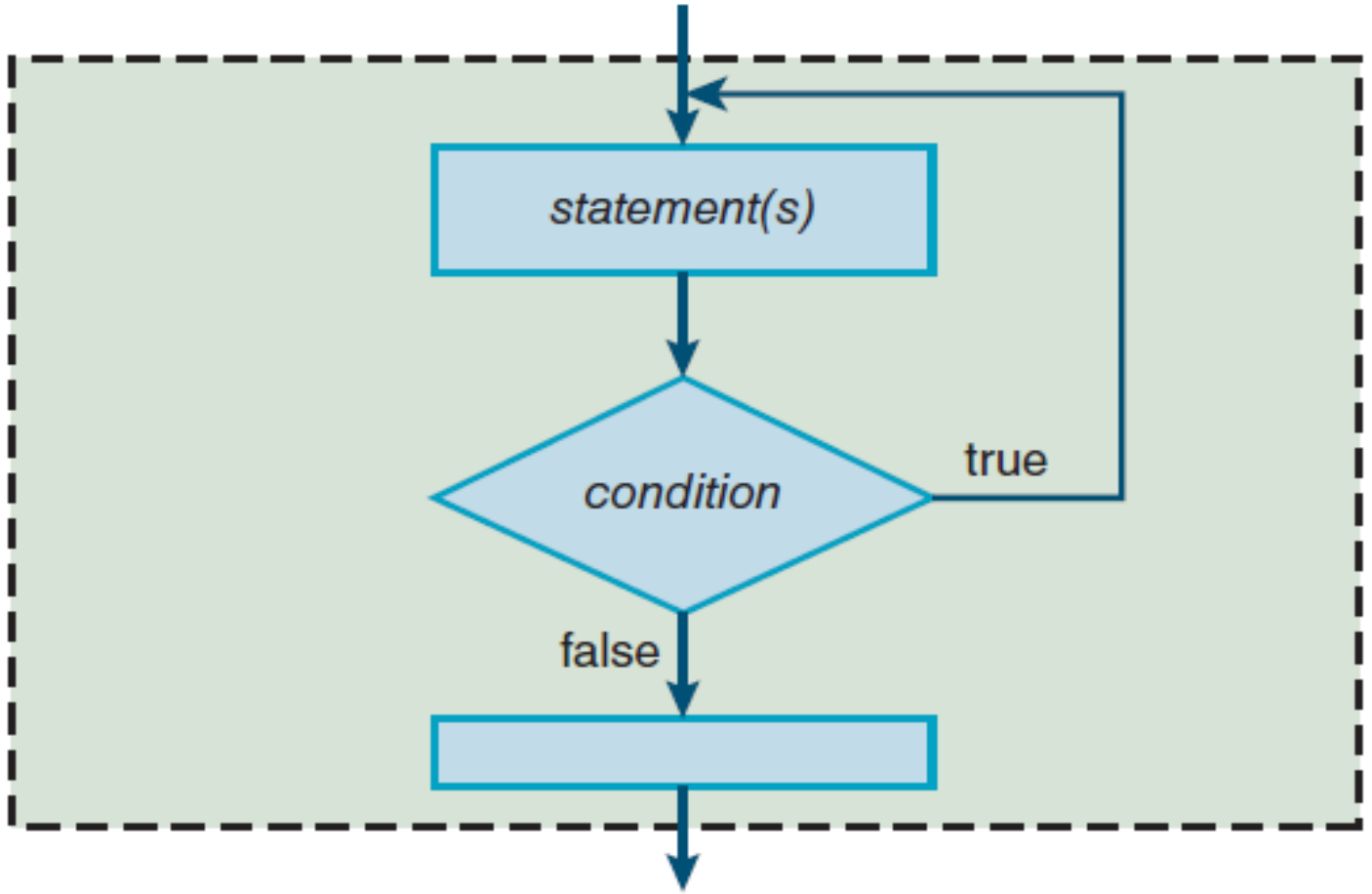


a.js

```
function factorial(n) {
    var ans = 1,i=1;
      if(n === 0)
        return 1;
    while(i<=n)
          {
       ans = ans * i;
        i++;
          }
    return ans;
}
 document.write(factorial(5));
```

do Loop

```
do {
    statement;
    statement;
    ...
    statement;
} while (condition) ;
```



Syntax and semantics for the `do` loop

do while Loop

```
<html><body>
<script>
function factorial(n) {
    var ans = 1,i=1;
      if(n === 0)
        return 1;
   do
           {
        ans = ans * i;
         i++;
           } while(i<=n);
    return ans;
}
 document.write(factorial(4));
</script>
</body></html>
```

Radio buttons

HTML Attributes

Here are the radio button control's more important attributes:

Radio Button Control Attributes						
type	name	value	checked	required	disabled	onclick

To group radio buttons together, the radio buttons have a **name attribute with the same value.**

same name

causes the radio button to be preselected

```
<input type="radio" name="color" value="black" checked>Black
  
<input type="radio" name="color" value="pistachio">Pistachio
  
<input type="radio" name="color" value="indigoBlue">Indigo Blue
```

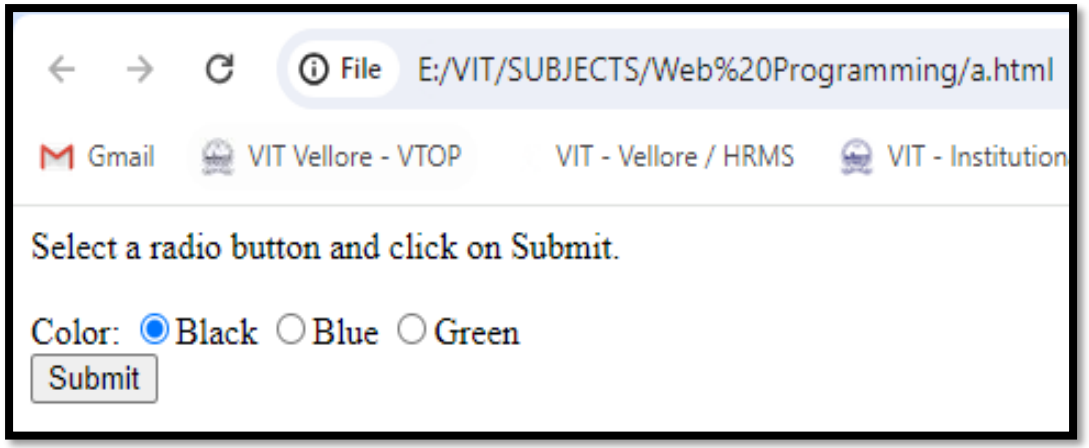
Properties for radio button collections and for individual radio buttons

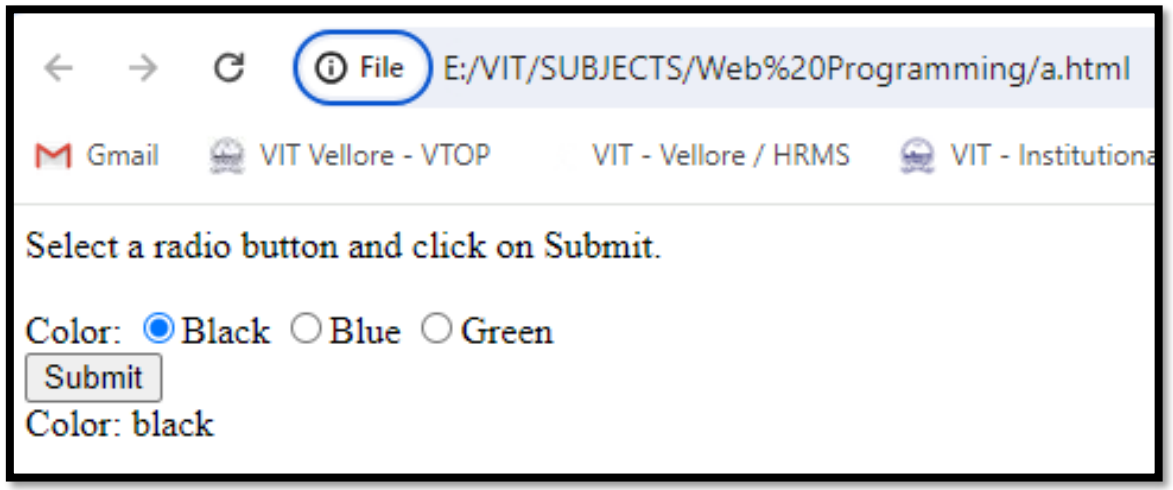
radio-button-collection.`value` Returns/assigns the selected radio button's value.
radio-button-collection.`length` Returns the number of buttons in the radio button collection.
radio-button-collection[*i*].`value` Returns/assigns the *i*th button's value.
radio-button-collection[*i*].`defaultChecked` Returns true or false, for whether the *i*th button was preselected.
radio-button-collection[*i*].`checked` Returns/assigns true or false, for whether the *i*th button is currently selected.
radio-button-collection[*i*].`disabled` Returns/assigns true or false, for whether the *i*th button is disabled.

Using JavaScript to Retrieve Radio Button Objects

```
<html>
<head> <title> How to get value of selected radio button using JavaScript ?
        </title>
</head>
<body>
        <p> Select a radio button and click on Submit.</p>
        Color:
        <input type="radio" name="color" value="black" checked>Black
        <input type="radio" name="color" value="blue">Blue
        <input type="radio" name="color" value="green">Green
        <br>
        <button type="button" onclick="displayRadioValue()">Submit</button>
        <br>
        <div id="result"></div>
```

```
<script>
function displayRadioValue() {
 var ele = document.getElementsByName('color');
for (i = 0; i < ele.length; i++)
 {
if (ele[i].checked)
document.getElementById("result").innerHTML= "Color: " + ele[i].value;
}
}
</script>
</body>
</html>
```





Checkboxes

HTML Attributes

Here are the checkbox control's more important attributes:

Checkbox Control Attributes							
type	id	name	value	checked	required	disabled	onclick

To group check boxes together, the checkboxes have a **name attribute with the same value.**

same name

causes the checkbox to be preselected

```
<input type="checkbox" name="jobSkills" value="HTML5" checked>
  know HTML5<br>
<input type="checkbox" name="jobSkills" value="CSS"> know CSS<br>
<input type="checkbox" name="jobSkills" value="coffee making">
  make good coffee<br><br>
```

- [x] know HTML5
- [ ] know CSS
- [ ] make good coffee

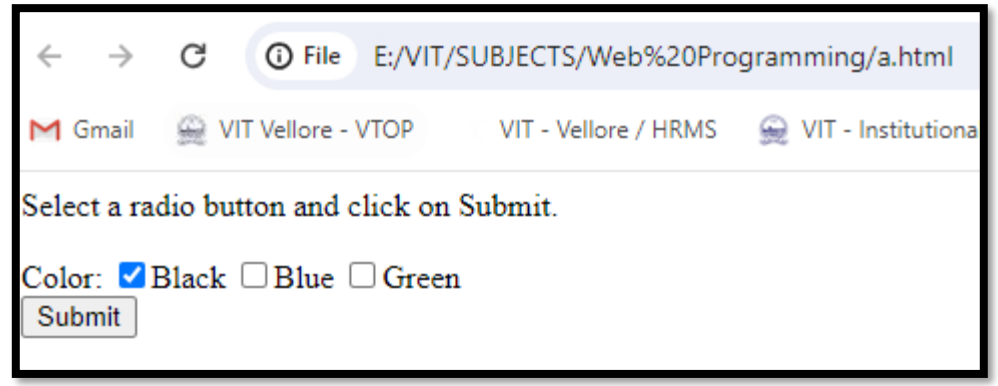
Properties for checkbox collections and for individual collections

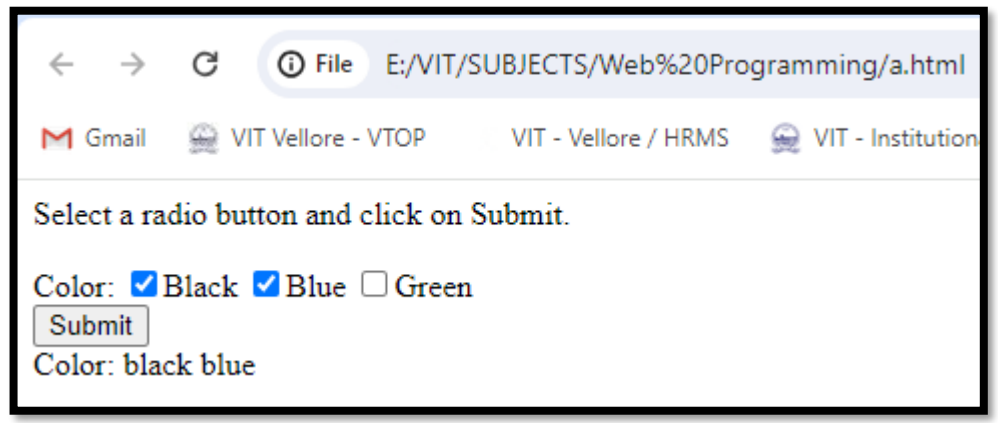
checkbox-collection.`length` Returns the number of buttons in the checkbox collection.
checkbox.`value` Assigns/returns the checkbox's value.
checkbox.`defaultChecked` Returns true or false, for whether the checkbox was preselected.
checkbox.`checked` Assigns/returns true or false, for whether the checkbox is currently selected.
checkbox.`disabled` Assigns/returns true or false, for whether the checkbox is disabled.

Using JavaScript to Retrieve Check box Objects

```
<html>
<head><title> How to get value of selected radio button using JavaScript ? </title>
</head>
<body> <p>Select a radio button and click on Submit.</p>
Color:
<input type="checkbox" name="color" value="black" checked>Black
<input type="checkbox" name="color" value="blue">Blue
<input type="checkbox" name="color" value="green">Green
<br>
<button type="button" onclick="displayCheckValue()">
Submit
</button>
<br>
<div id="result"></div>
```

```
<script>
function displayCheckValue() {
 var ele = document.getElementsByName('color');
var result="";
for (i = 0; i < ele.length; i++) {
if (ele[i].checked)
result += ele[i].value+" "
}
document.getElementById("result").innerHTML= "Color: " + result;
}
</script>
</body>
</html>
```





for Loop

```
for (initialization; condition; update) {
```

1. Initialization component

   Before the first pass through the body of the loop, execute the initialization component.
2. Condition component

   Before each loop iteration, evaluate the condition component:
   - If the condition is true, execute the body of the loop.
   - If the condition is false, terminate the loop (exit to the statement below the loop's closing brace).
3. Update component

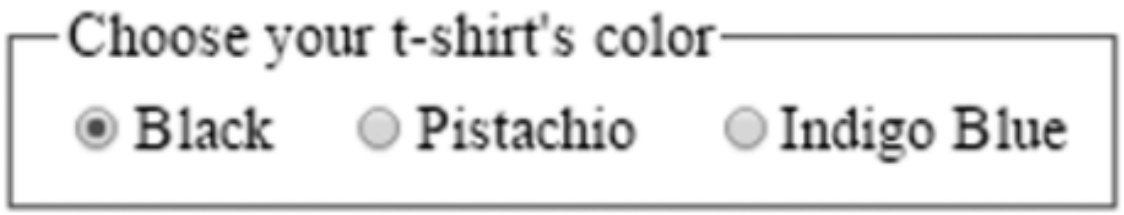
   After each pass through the body of the loop, return to the loop heading and execute the update component. Then, recheck the continuation condition in the second component, and if it's satisfied, go through the body of the loop again.

for Loop

```
<html><body>
<script>
function factorial(n) {
    var ans = 1;
      if(n === 0)
        return 1;
   for(var i=1;i<=n;i++)
          {
        ans = ans * i;
          }
    return ans;
}
 document.write(factorial(4));
</script>
</body></html>
```

fieldset and legend Elements

- The <fieldset> tag is used to group related elements in a form.
- It draws a box around the related elements.
- <legend> tag is used to define a caption for the <fieldset> element




```
<fieldset>
<legend>Choose your t-shirt's color</legend>
<input type="radio" name="color" value="black">Black   
<input type="radio" name="color" value="pistachio">Pistachio  
<input type="radio" name="color" value="indigoBlue">Indigo Blue
</fieldset>
```

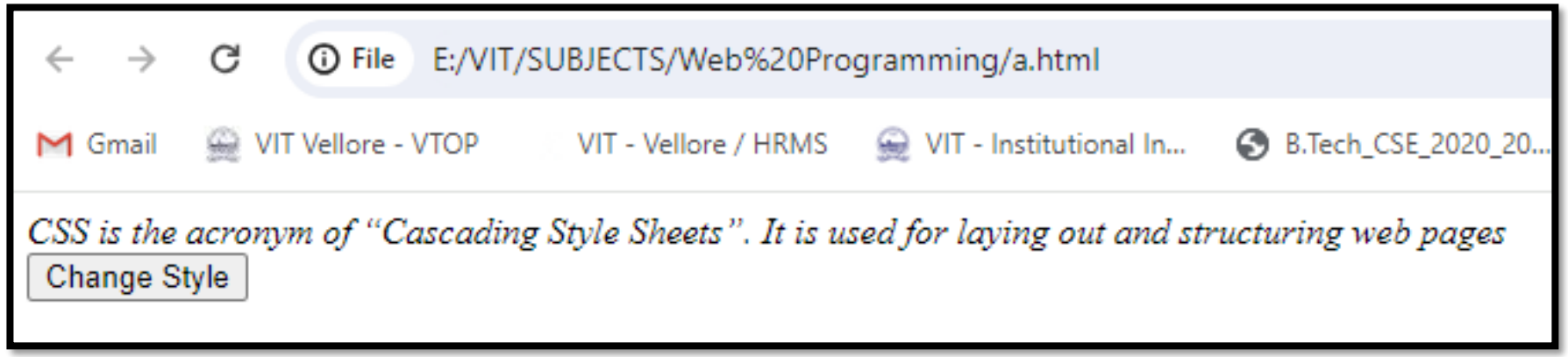
Fieldset and legend

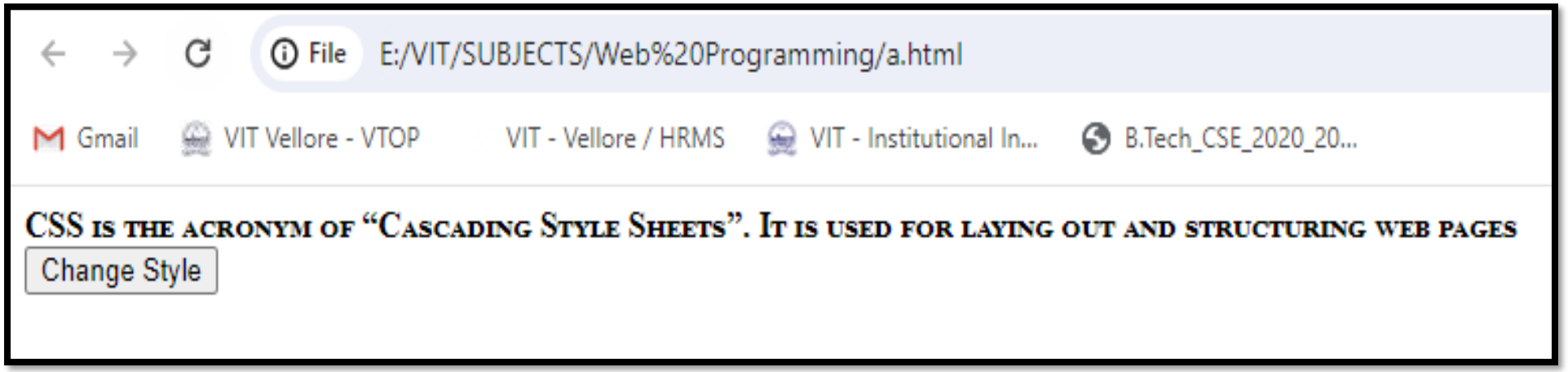
```
<html><head>
<style>
fieldset {display: inline;}
</style></head>
<body>
<h1>The fieldset and legend element</h1>
<form>
 <fieldset>
  <legend>Personalia:</legend>
  <label for="fname">First name:</label>
  <input type="text" id="fname" name="fname"><br><br>
  <label for="lname">Last name:</label>
  <input type="text" id="lname" name="lname"><br><br>
   </fieldset>
</form>
</body></html>
```

Manipulating CSS with JavaScript-

1. Assigning a New Value to an Element's class Attribute

```
<html>
<head><style>
.italic {font-style: italic;}
.bold {font-weight: bold;}
.small-caps {font-variant: small-caps;}
</style></head>
<body>
<div class="italic" id="message">
CSS is the acronym of "Cascading Style
Sheets". It is used for laying out and
structuring web pages</div>
<button onClick="change()">Change Style </button>
<script>
function change()
{
document.getElementById("message").className =
                                    "bold small-caps";
}
</script>
</body>
</html>
```

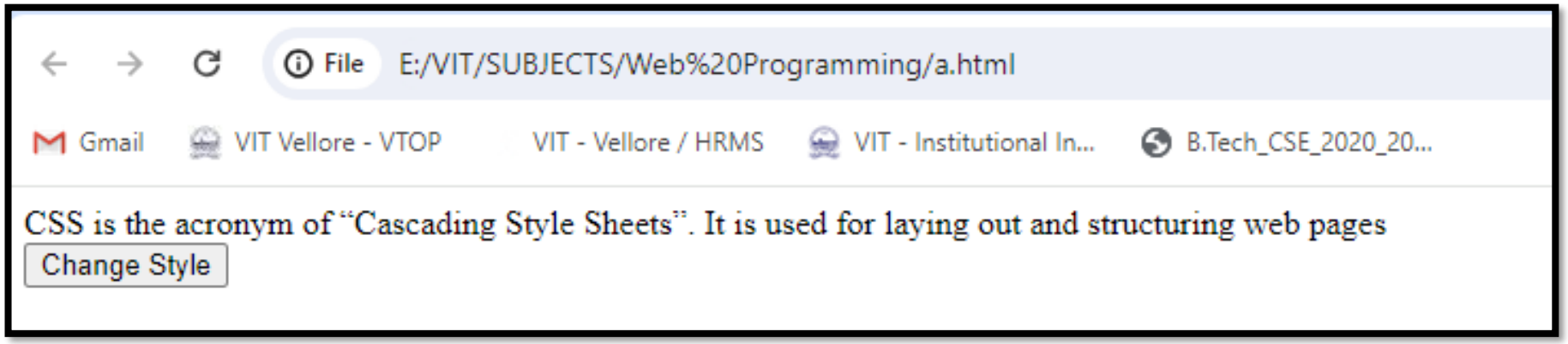

File
E:/VIT/SUBJECTS/Web%20Programming/a.html
Gmail
VIT Vellore - VTOP
VIT - Vellore / HRMS
VIT - Institutional In...
B.Tech_CSE_2020_20...
CSS is the acronym of "Cascading Style Sheets". It is used for laying out and structuring web pages
Change Style


File
E:/VIT/SUBJECTS/Web%20Programming/a.html
Gmail
VIT Vellore - VTOP
VIT - Vellore / HRMS
VIT - Institutional In...
B.Tech_CSE_2020_20...
CSS IS THE ACRONYM OF "CASCADING STYLE SHEETS". IT IS USED FOR LAYING OUT AND STRUCTURING WEB PAGES
Change Style

Manipulating CSS with JavaScript-

2. Assigning a Value to an Element's style Property

```
<html><body>
<div id="message">
CSS is the acronym of "Cascading Style Sheets". It is used for laying out and structuring
web pages
</div>
<button onClick="change()">Change Style </button>
<script>
function change()
{
document.getElementById("message").style.fontSize = "3em";
document.getElementById("message").style.color = "red";
}
</script></body>
</html>
```


File
E:/VIT/SUBJECTS/Web%20Programming/a.html
Gmail
VIT Vellore - VTOP
VIT - Vellore / HRMS
VIT - Institutional In...
B.Tech_CSE_2020_20...
CSS is the acronym of "Cascading Style Sheets". It is used for laying out and structuring web pages
Change Style

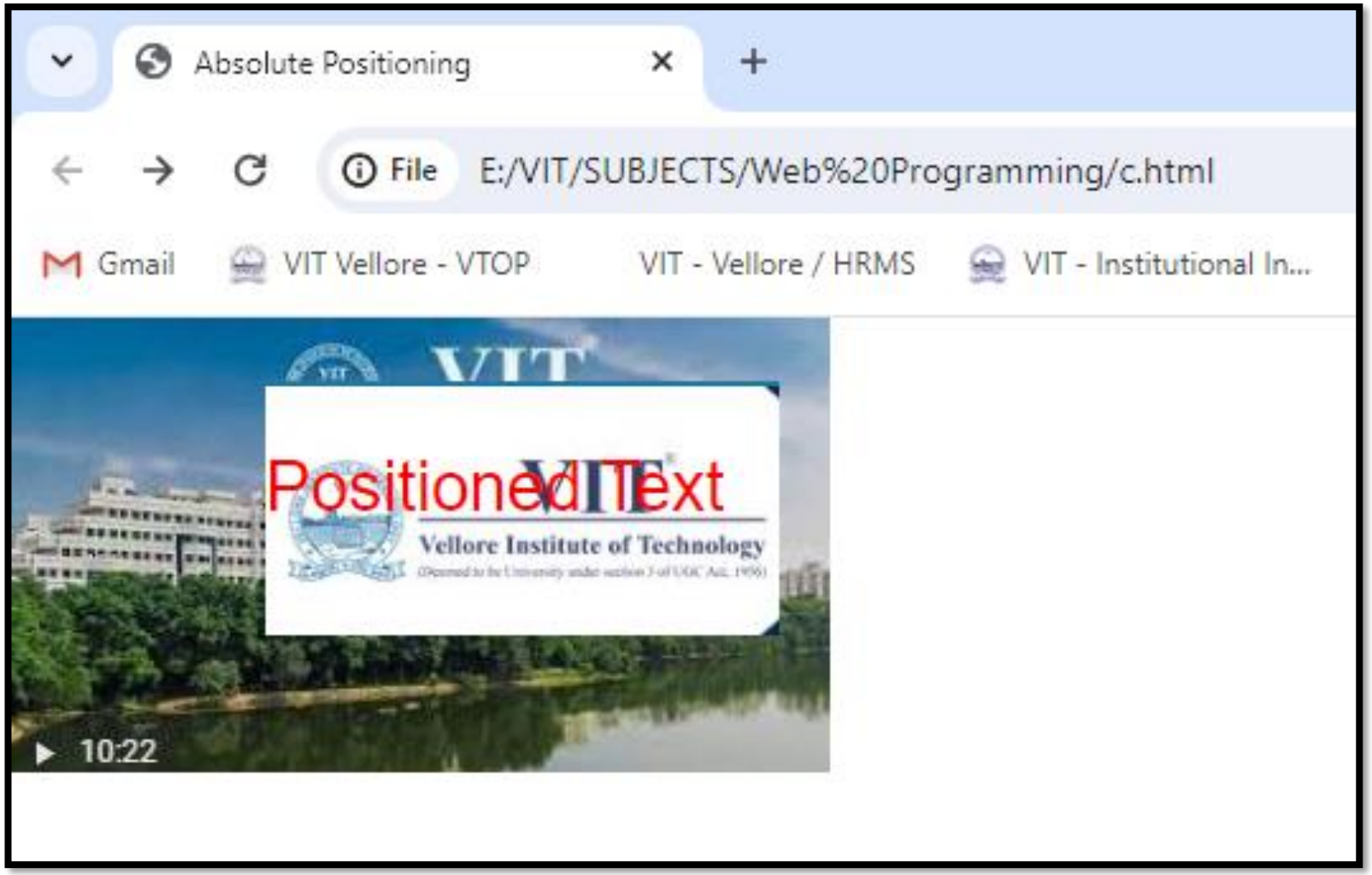

File
E:/VIT/SUBJECTS/Web%20Programming/a.html
Gmail
VIT Vellore - VTOP
VIT - Vellore / HRMS
VIT - Institutional In...
B.Tech_CSE_2020_20...
All Bookmarks
CSS is the acronym of "Cascading Style Sheets". It is used for laying out and structuring web pages
Change Style

Using z-index to Stack Elements

z-index Property

- The **z-index** property specifies the stack order of an element.
- An element with greater stack order is always in front of an element with a lower stack order

```
<!DOCTYPE html>
<html>
<head>
<meta charset = "utf-8">
<title>Absolute Positioning</title>
<style type = "text/css">
.image1
{
position: absolute;
top: 0px;
left: 0px;
z-index:1;
}
.image2
{
position: absolute;
top: 25px;
left: 100px;
z-index:2;
}
.text
 {
position: absolute;
top: 25px;
left: 100px;
z-index:3;
font-size: 20pt;
font-family:Arial;
color:red;
}
</style>
</head>
<body>
<img src="vit.jpg" class="image1"/>
<img src="logo.jpg" class="image2"/>
<p class = "text">Positioned Text</p>
</body>
</html>
```

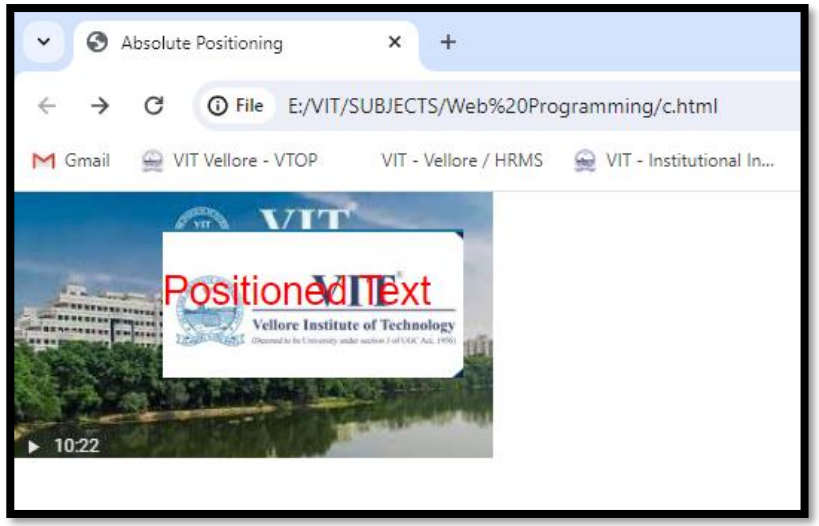

Absolute Positioning
File
E:/VIT/SUBJECTS/Web%20Programming/c.html
Gmail
VIT Vellore - VTOP
VIT - Vellore / HRMS
VIT - Institutional In...
VIT
PositionedText
Vellore Institute of Technology
10:22

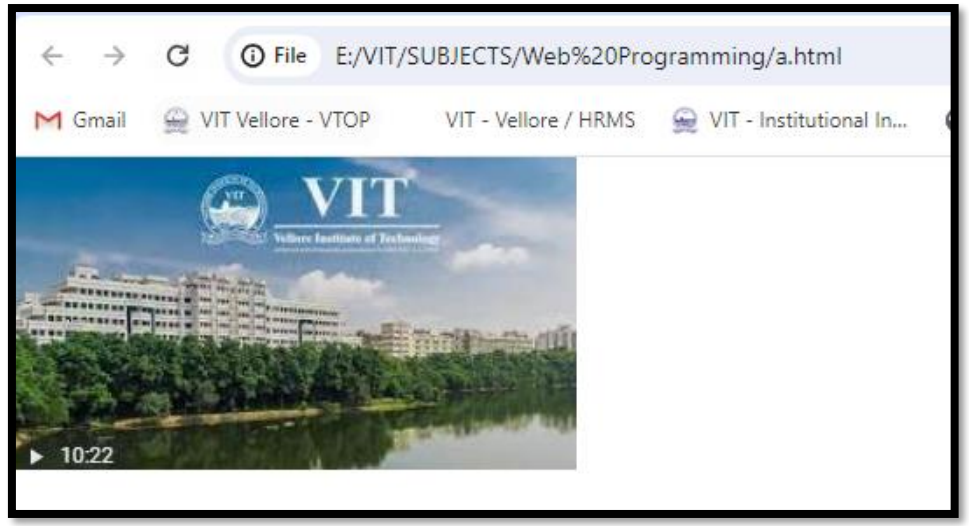
1. Using JavaScript to Modify the Stacking Order

```
<!DOCTYPE html>
<html>
<head>
<meta charset = "utf-8">
<title>Absolute Positioning</title>
<style type = "text/css">
.image1
{
position: absolute;
top: 0px;
left: 0px;
z-index:1;
}
.image2
{
position: absolute;
top: 25px;
left: 100px;
z-index:2;
}
.text
 {
position: absolute;
top: 25px;
left: 100px;
z-index:3;
font-size: 20pt;
font-family:Arial;
color:red;
}</style></head>
<body>
<img src="vit.jpg" class="image1" onclick="moveToTop(this)"/>
<img src="logo.jpg" class="image2" onclick="moveToTop(this)"/>
<p class = "text">Positioned Text</p>
<script>
var topIndex = 3;
function moveToTop(picture) {
picture.style.zIndex = ++topIndex;
}</script>
</body></html>
```

Note:
this refers to the clicked image's object

The picture parameter receives the clicked image's object.


Absolute Positioning
File E:/VIT/SUBJECTS/Web%20Programming/c.html
Gmail
VIT Vellore - VTOP
VIT - Vellore / HRMS
VIT - Institutional In...
Positioned Text
VIT
Vellore Institute of Technology
10:22


File E:/VIT/SUBJECTS/Web%20Programming/a.html
Gmail
VIT Vellore - VTOP
VIT - Vellore / HRMS
VIT - Institutional In...
VIT
Vellore Institute of Technology
10:22


File E:/VIT/SUBJECTS/Web%20Programming/a.html
Gmail
VIT Vellore - VTOP
VIT - Vellore / HRMS
VIT - Institutional
VIT
Vellore Institute of Technology
10:22

Textarea Controls

- The textarea control is similar to the text control in that it enables the user to enter text into a box.

HTML Attributes

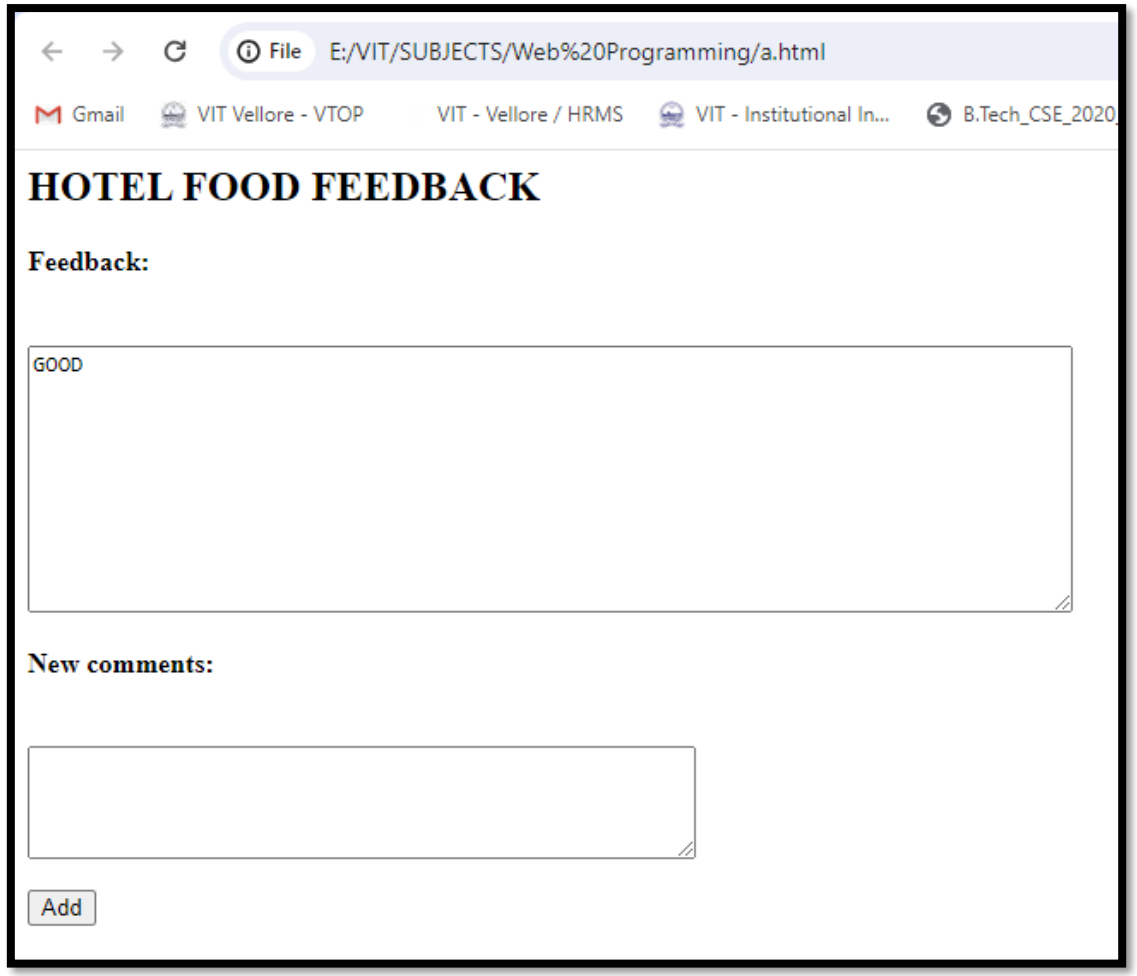
Here are the textarea control's more important attributes:

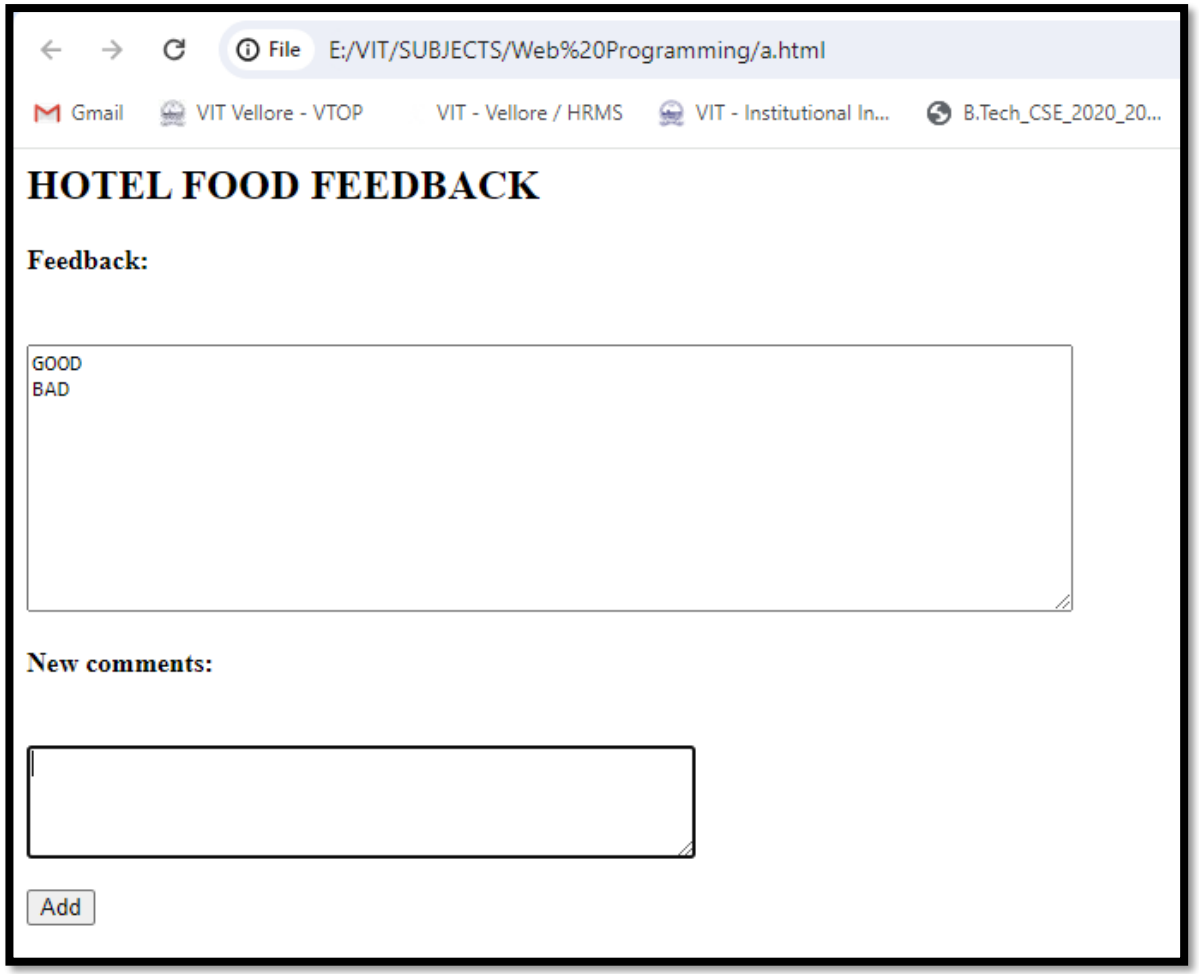
Textarea Control Attributes								
id	rows	cols	maxlength	placeholder	spellcheck	disabled	readonly	required

1. Using JavaScript with the Textarea Control

```
<html>
<body>
   <h2>HOTEL FOOD FEEDBACK</h2>
   <form>
      <h4>Feedback:</h4><br>
      <textarea id="allComments" rows="10" cols="80" readonly>GOOD</textarea><br>
      <h4>New comments:</h4><br>
      <textarea id="newComment" rows="4" cols="50" required
spellcheck="true"></textarea><br><br>
      <input type="button" value="Add" onclick="addComment(this.form);">
   </form>
```

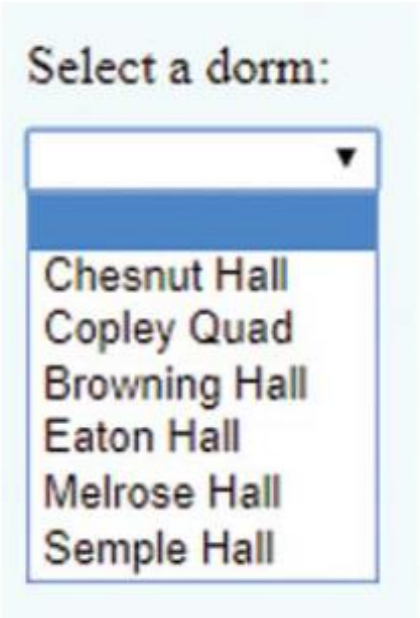
```
<script>
        function addComment(form) {
            var newComment = form.elements["newComment"];
            var allComments = form.elements["allComments"].value;
            document.getElementById("allComments").value = allComments + '\n' +
newComment.value;
            newComment.value = "";
        }
    </script>
</body>
</html>
```

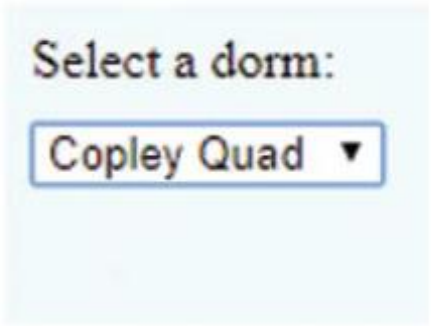

File E:/VIT/SUBJECTS/Web%20Programming/a.html
Gmail VIT Vellore - VTOP VIT - Vellore / HRMS VIT - Institutional In... B.Tech_CSE_2020
HOTEL FOOD FEEDBACK
Feedback:
GOOD
New comments:
Add


File E:/VIT/SUBJECTS/Web%20Programming/a.html
Gmail VIT Vellore - VTOP VIT - Vellore / HRMS VIT - Institutional In... B.Tech_CSE_2020_20...
HOTEL FOOD FEEDBACK
Feedback:
GOOD
BAD
New comments:
Add

Pull-Down Menus

- When a user clicks a pull-down menu's down arrow, that causes a list of selections to appear below the down arrow.
- When the user clicks one of the selections, the list collapses and the selected value appears next to the down arrow.






```
<select id="dorm" required>
  <option label=" "></option>
  <option>Chesnut Hall</option>
  <option>Copley Quad</option>
  <option>Browning Hall</option>
  <option>Eaton Hall</option>
  <option>Melrose Hall</option>
  <option>Semple Hall</option>
</select>
```

HTML Attributes

Typically, `select` element syntax is sparse. Just a few attributes are commonly used with the `select` element for pull-down menus, and here they are:

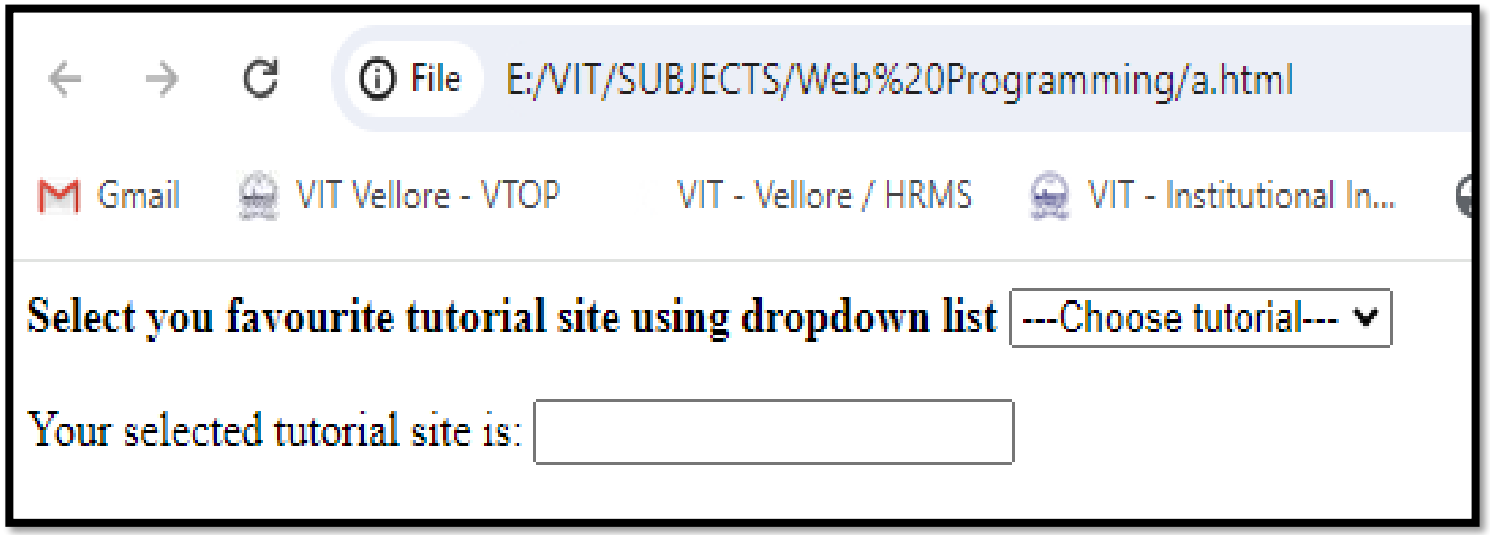
Pull-Down Menu `select` Element Attributes		
`id`	`disabled`	`required`

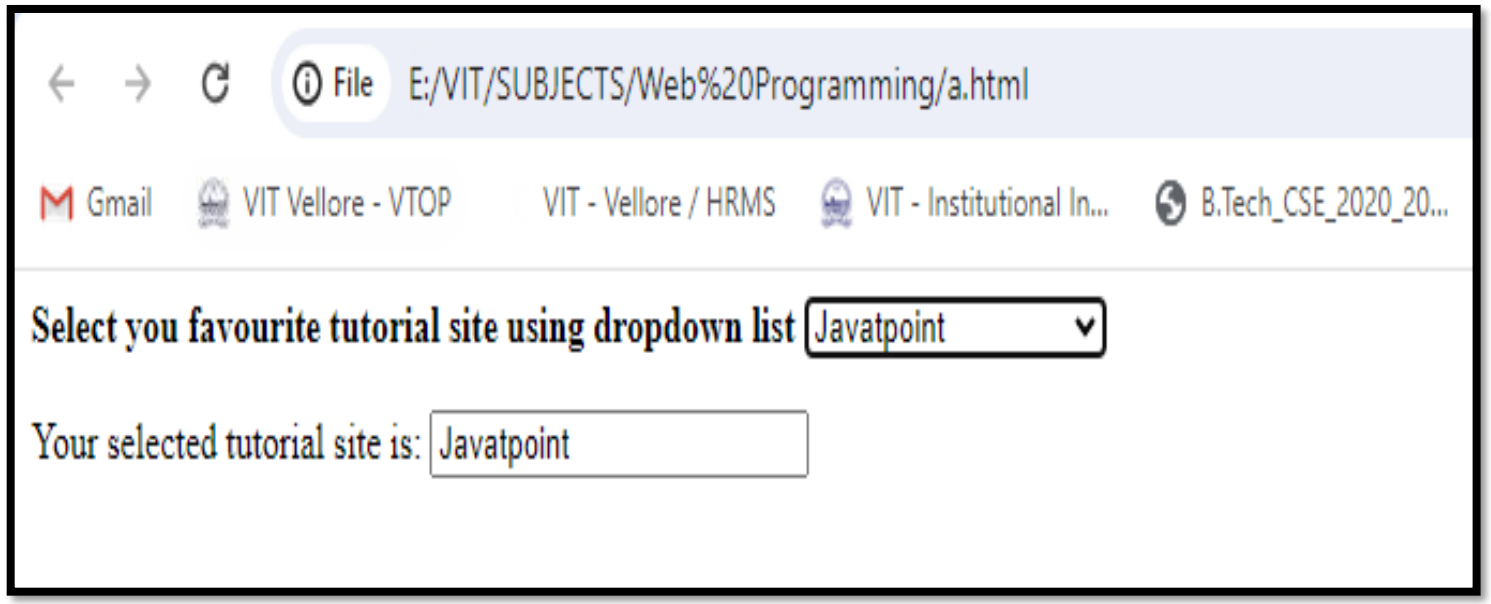
Pull-Down Menu `option` Element Attributes		
`label`	`selected`	`value`

1. Using JavaScript with Pull-Down Menus

```
<html>
<head>
<title>dropdown menu using select tab</title>
</head>
<script>
function favTutorial() {
var mylist = document.getElementById("myList");
document.getElementById("favourite").value =
mylist.options[mylist.selectedIndex].text;
}
</script>
```

```
 <body>
<form>
<b> Select you favourite tutorial site using dropdown list </b>
<select id = "myList" onchange = "favTutorial()" >
<option> ---Choose tutorial--- </option>
<option> w3schools </option>
<option> Javatpoint </option>
<option> tutorialspoint </option>
<option> geeksforgeeks </option>
</select>
<p> Your selected tutorial site is:
<input type = "text" id = "favourite" size = "20“>
</form>
</body>
</html>
```


File
E:/VIT/SUBJECTS/Web%20Programming/a.html
Gmail
VIT Vellore - VTOP
VIT - Vellore / HRMS
VIT - Institutional In...
Select you favourite tutorial site using dropdown list
---Choose tutorial---
Your selected tutorial site is:

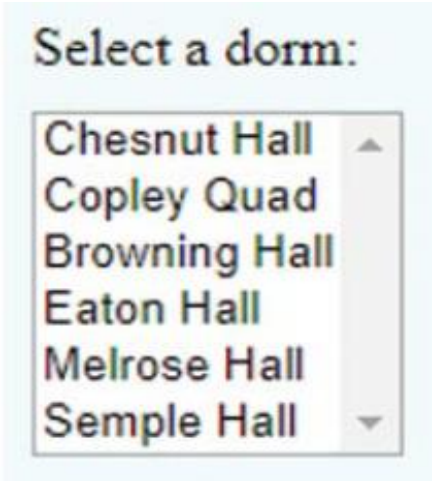

File
E:/VIT/SUBJECTS/Web%20Programming/a.html
Gmail
VIT Vellore - VTOP
VIT - Vellore / HRMS
VIT - Institutional In...
B.Tech_CSE_2020_20...
Select you favourite tutorial site using dropdown list
Javatpoint
Your selected tutorial site is:
Javatpoint

List Boxes

- The List boxes are similar to pull-down menus
- The pull-down menus collapse after the user selects an option. With a list box, the list of options remains viewable after the user selects an option.

size="6" six options display simultaneously in the list box.
If there's **no size attribute**, then the default size is 4.

That's the same code as for the pull-down menu, except we've added multiple and size attributes




```
<select id="dorm" multiple size="6" required>
 <option>Chesnut Hall</option>
 <option>Copley Quad</option>
 <option> Browning Hall</option>
 <option> Eaton Hall</option>
 <option> Melrose Hall</option>
 <option> Semple Hall</option>
</select>
```

HTML Attributes

Here are the attributes that are commonly used with the `select` element for list boxes:

List Box `select` Element Attributes				
`id`	`disabled`	`required`	`multiple`	`size`

List Box `option` Element Attributes		
`label`	`selected`	`value`

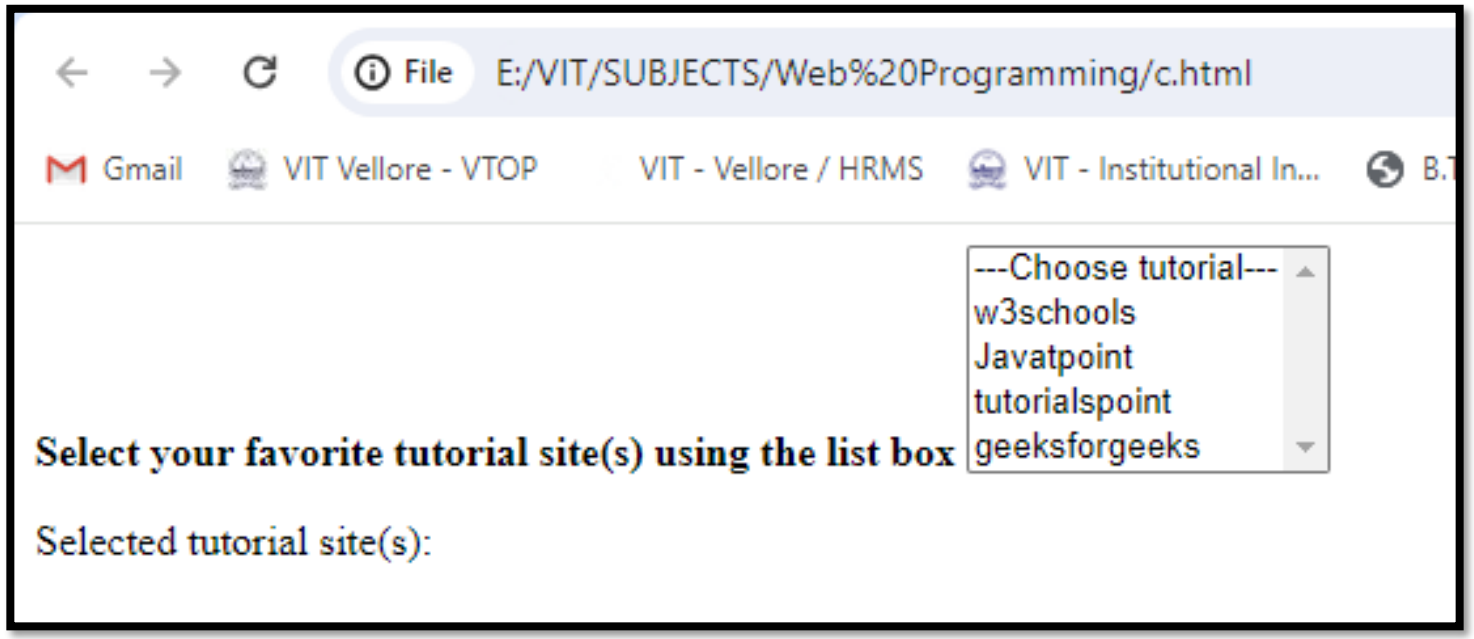
Using JavaScript with List Boxes

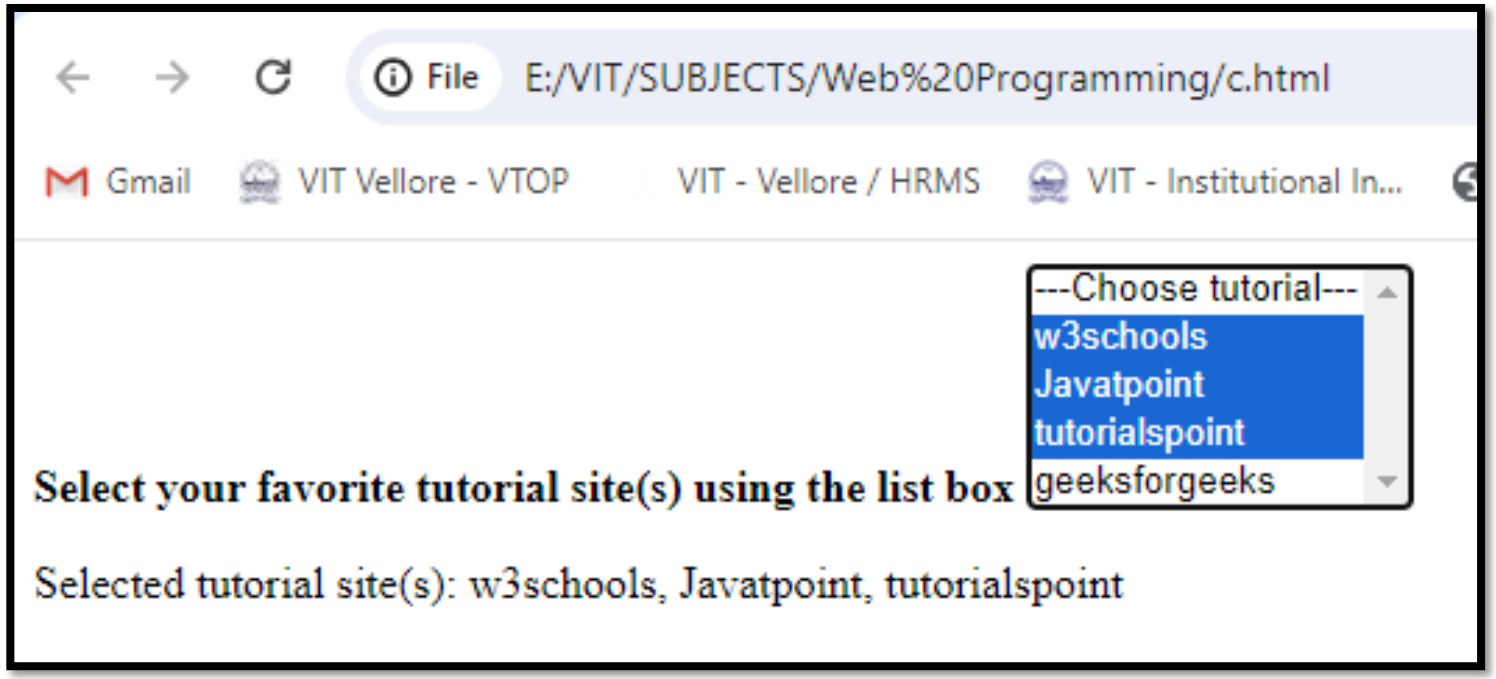
Object	Property	Description
`select` element's object	`selectedOptions`	Retrieve the collection of options that are currently selected in the list box.
`option` element's object	`value`	Access the option's value.

1. Using JavaScript with Listbox

```
<html><head>
   <title>List Box Using selectedOptions</title>
</head>
<script>
   function favTutorial() {
      var myList = document.getElementById("myList");
      var selectedOptions = myList.selectedOptions;
      var selectedText = "";
      for (var i = 0; i < selectedOptions.length; i++) {
         selectedText += selectedOptions[i].text;
         if (i < selectedOptions.length - 1) {
            selectedText += ", ";
         }      }
      document.getElementById("selectedResult").innerText = selectedText
   }</script>
```

```
<body>
   <form>
      <b>Select your favorite tutorial site(s) using the list box</b>
      <select id="myList" multiple size="5" onchange="favTutorial()">
         <option>---Choose tutorial---</option>
         <option value="w3schools">w3schools</option>
         <option value="Javatpoint">Javatpoint</option>
         <option value="tutorialspoint">tutorialspoint</option>
         <option value="geeksforgeeks">geeksforgeeks</option>
      </select>
      <p>Selected tutorial site(s): <span id="selectedResult"> </span></p>
   </form>
</body>
</html>
```


File
E:/VIT/SUBJECTS/Web%20Programming/c.html
Gmail
VIT Vellore - VTOP
VIT - Vellore / HRMS
VIT - Institutional In...
---Choose tutorial---
w3schools
Javatpoint
tutorialspoint
geeksforgeeks
Select your favorite tutorial site(s) using the list box
Selected tutorial site(s):


File
E:/VIT/SUBJECTS/Web%20Programming/c.html
Gmail
VIT Vellore - VTOP
VIT - Vellore / HRMS
VIT - Institutional In...
---Choose tutorial---
w3schools
Javatpoint
tutorialspoint
geeksforgeeks
Select your favorite tutorial site(s) using the list box
Selected tutorial site(s): w3schools, Javatpoint, tutorialspoint

Canvas and Drawing

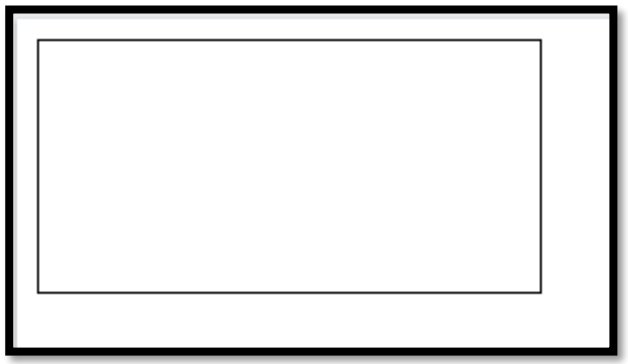
- To use canvas,

  (1) a canvas element and

  (2) JavaScript method calls that draw graphics objects within the canvas element's drawing area.

- The canvas element is used to draw graphics, on the fly, via JavaScript.
- The canvas element is only a container for graphics. use JavaScript to actually draw the graphics.
- Canvas has several methods for drawing paths, boxes, circles, text, and adding images.

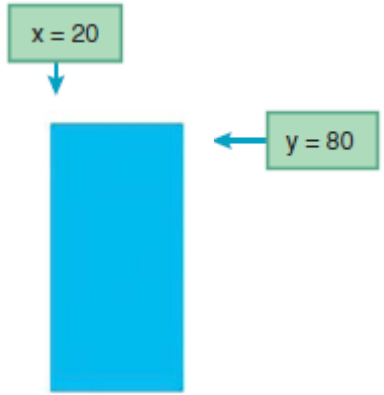
Empty canvas



```
<!DOCTYPE html>
<html>
<body>
<canvas id="myCanvas" width="200" height="100" style="border:1px
solid #000000;">
Your browser does not support the HTML canvas tag.
</canvas>
</body>
</html>
```

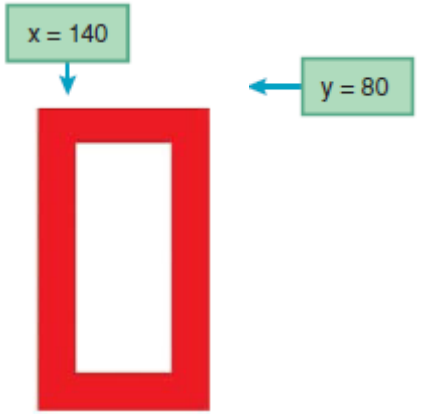
Rectangles Web Page

- To draw a filled-in rectangle, use the context object to call the fillRect method. Here's the syntax: context.fillRect(x, y, width, height);
- Fill Color →
  - ctx.fillStyle = "deepskyblue";
  - ctx.fillRect(20, 80, 70, 140);



- color value formats→
  - color name
  - RGB value—specifies amounts of red, green, and blue
  - RGBA value—specifies RGB, plus the amount of opacity
  - HSL value—specifies amounts of hue, saturation, and lightness
  - HSLA—specifies HSL, plus the amount of opacity

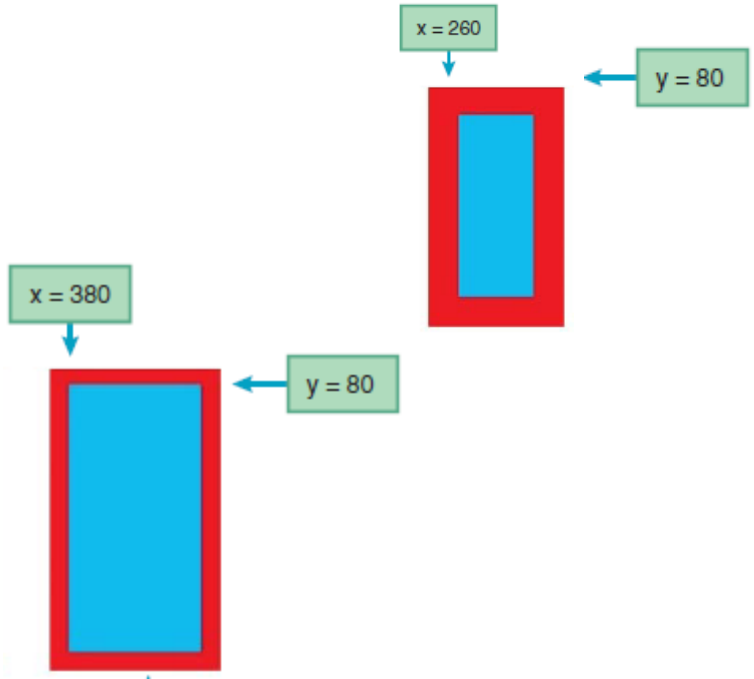
- Rectangle Borders→
  - context.strokeRect(x, y, width, height);
  - ctx.lineWidth = 20; (default width is 1 pixel)
  - ctx.strokeStyle = "red";(default drawing color is black)



- A rectangle, which has different colors for its border and interior
  - Case 1: ctx.fillRect(260, 80, 70, 140);

    ctx.strokeRect(260, 80, 70, 140);
  - Case 2: ctx.strokeRect(260, 80, 70, 140);

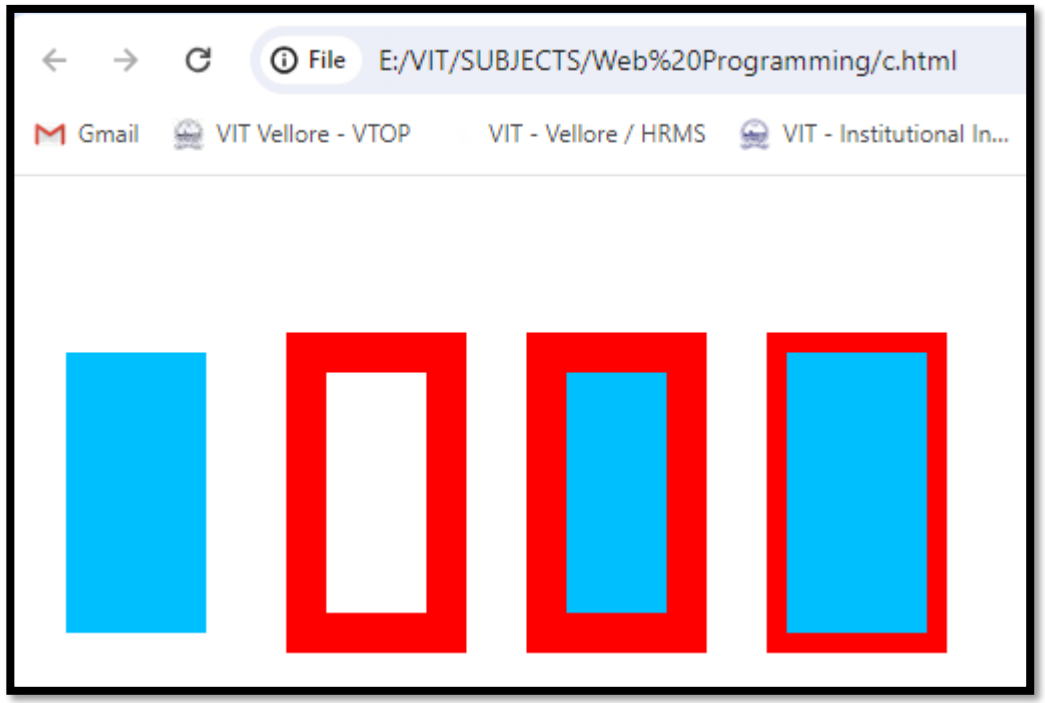
    ctx.fillRect(260, 80, 70, 140);



Example:

```
<!DOCTYPE html>
<head>
<title>Rectangle Examples</title>
<script>
function rectangleExamples() {
var ctx; // the canvas object's context
ctx = document.getElementById("canvas").getContext("2d");
ctx.fillStyle = "deepskyblue";
ctx.fillRect(20, 80, 70, 140);
ctx.lineWidth = 20;
ctx.strokeStyle = "red";
ctx.strokeRect(140, 80, 70, 140);
ctx.fillRect(260, 80, 70, 140);
ctx.strokeRect(260, 80, 70, 140);
```

```
ctx.strokeRect(380, 80, 70, 140);
ctx.fillRect(380, 80, 70, 140);
} // end rectangleExamples
</script>
</head>
<body onload="rectangleExamples();">
<canvas id="canvas" width="480" height="250">
Sorry - This page uses <code>canvas</code>, and
your browser doesn't support it.
</canvas>
</body>
</html>
```



Using a Color Gradient to Fill a Shape

- Syntax for retrieving a gradient object from the context object

  `context.createLinearGradient(x1, y1, x2, y2);`

  - The x1, y1 arguments form the coordinates for the point where you can specify the first color in your gradient, and the x2, y2 arguments form the coordinates for the point where you can specify the last color in your gradient

- To specify the colors, call the gradient object's addColorStop method one time for each color.

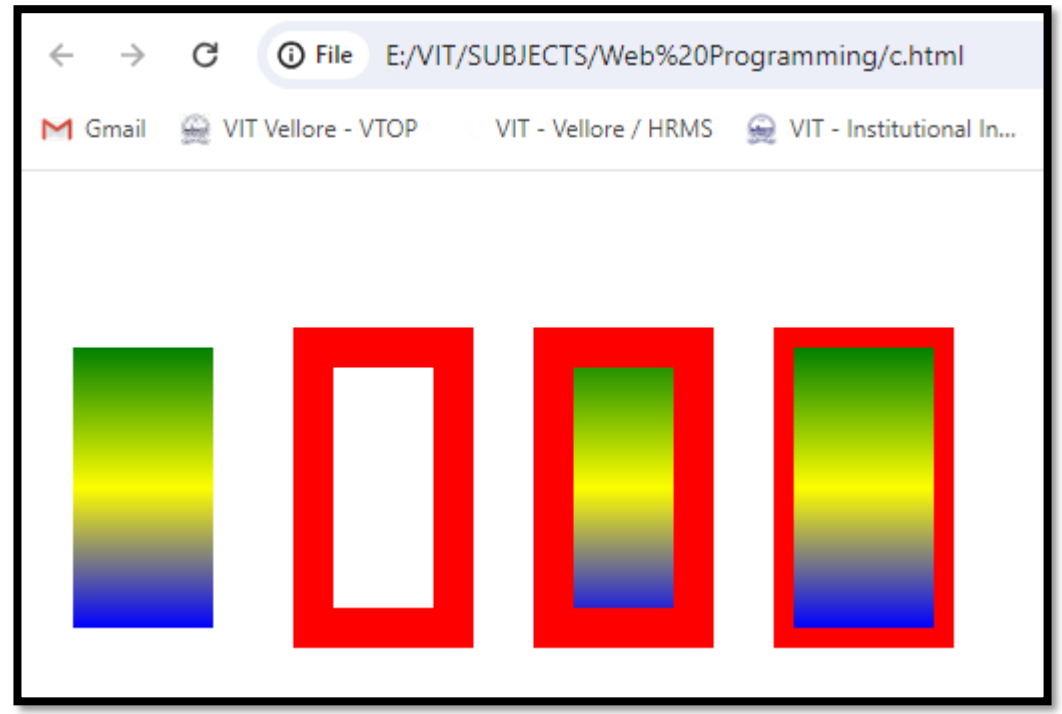
  Here's the syntax: `gradient.addColorStop(offset, color);`

  - The offset argument, a number between 0 and 1, specifies where the color is positioned within the gradient's range. A 0 value indicates that the color is at the starting point formed by x1, y1. A 1 value indicates that the color is at the ending point formed by x2

Example:

```
<!DOCTYPE html>
<head>
<title>Rectangle Examples</title>
  <script>
    function rectangleExamples() {
      var ctx; // the canvas object's context
      ctx = document.getElementById("canvas").getContext("2d");
      var gradient = ctx.createLinearGradient(0, 80, 0, 220); // Adjust the gradient coordinates
      gradient.addColorStop(0, "green");
      gradient.addColorStop(0.5, "yellow");
      gradient.addColorStop(1, "blue");
      ctx.fillStyle = gradient;
      ctx.fillRect(20, 80, 70, 140);
```

```
      ctx.lineWidth = 20;
      ctx.strokeStyle = "red";
      ctx.strokeRect(140, 80, 70, 140);
      ctx.fillStyle = gradient; // Reuse the same gradient
      ctx.fillRect(260, 80, 70, 140);
      ctx.strokeRect(260, 80, 70, 140);
      ctx.strokeRect(380, 80, 70, 140);
      ctx.fillRect(380, 80, 70, 140);
    } // end rectangleExamples
  </script>
</head>
<body onload="rectangleExamples();">
  <canvas id="canvas" width="480" height="250">
    Sorry - This page uses <code>canvas</code>, and your browser doesn't support it.  </canvas>
</body></html>
```



Drawing Text with fillText and strokeText

- To draw text character interiors, use the context object's fillText method.

  `context.fillText(text, x, y, max-width);`

  - The x and y parameters are coordinates for the placement of the lower-left corner of the text string within the canvas drawing area. The max- width parameter is optional, and it specifies the maximum width of the text argument's string.

- The strokeText method works the same as the fillText method except that it draws its text argument characters with borders, and it leaves the characters' interiors empty.

  `context.strokeText(text, x, y, max-width);`

Formatting Text

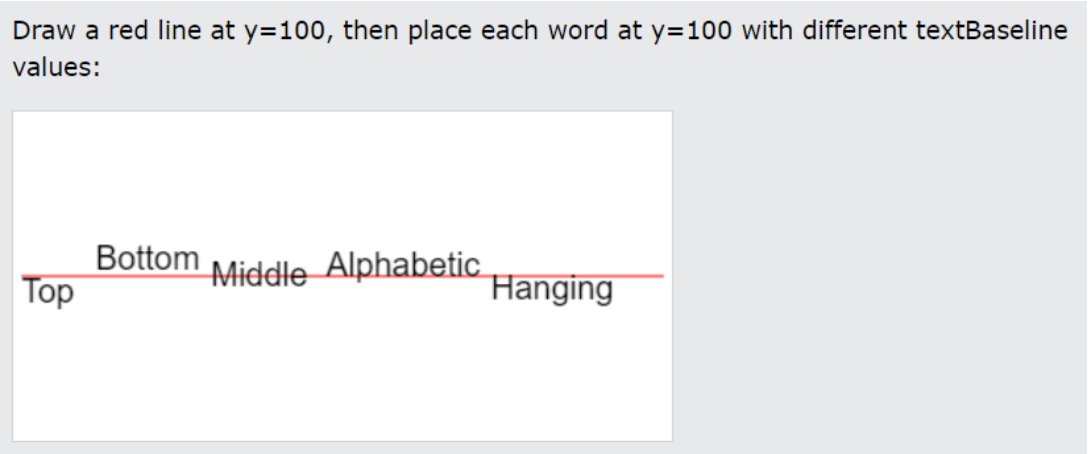
font Property

- context.font = [font-style-value] [font-variant-value] [font-weight-value] font-size-value[/line-height-value] font-family-value;
- Ex: ctx.font = "2em 'Times New Roman', serif";

textAlign property

- ctx.textAlign = "center";

textBaseline Property



- This property sets or returns the text baseline used when drawing text.
- Default value is alphabetic

Example:

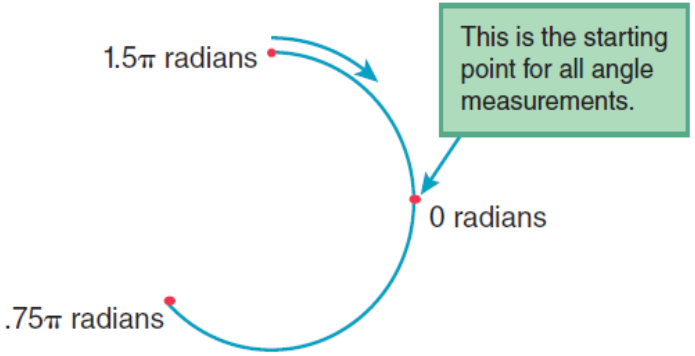
```
<!DOCTYPE html>
<head>
<title>George Orwell's 1984</title>
<script>
function displayQuotes() {
var ctx; // the canvas object's context
ctx = document.getElementById("canvas").getContext("2d");
ctx.font = "2em 'Times New Roman', serif";
ctx.fillText("\"The clocks were striking thirteen.\"", 5, 30);
ctx.textAlign = "center";
ctx.textBaseline = "top";
ctx.fillText("\"The clocks were striking thirteen.\"", 300, 110, 350);
ctx.strokeText("\"He loved Big Brother.\"", 300, 160, 350);
} </script>
```

```
</head>
<body onload="displayQuotes();">
<canvas id="canvas" width="600" height="170">
</canvas>
</body>
</html>
```

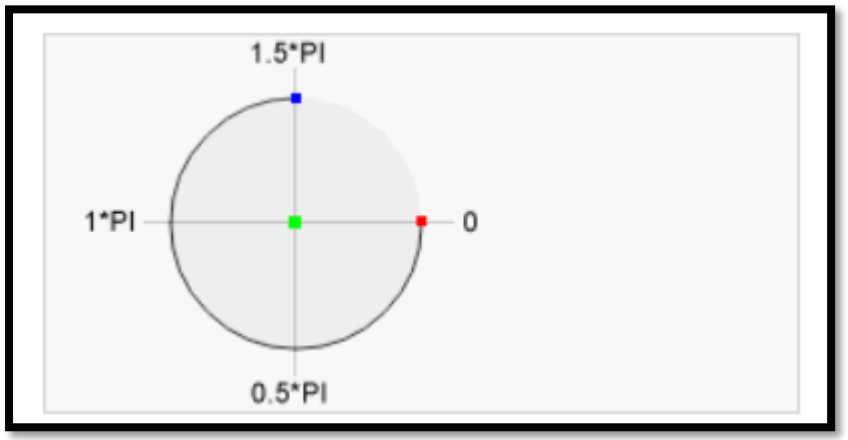
Drawing Arcs and Circles

False→ clockwise direction (default)
True→Counterclockwise

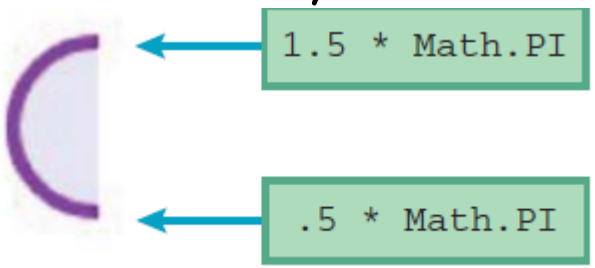
- *context.arc(x, y, radius, start-angle, end-angle, clockwise);*
- context.arc (x, y, radius, 1.5 * Math.PI, .75 * Math.PI);



- ctx.arc(80, 35, 25, 0, 2 * Math.PI); // circle creation



- ctx.arc(80, 35, 25, .5 * Math.PI, 1.5 * Math.PI);

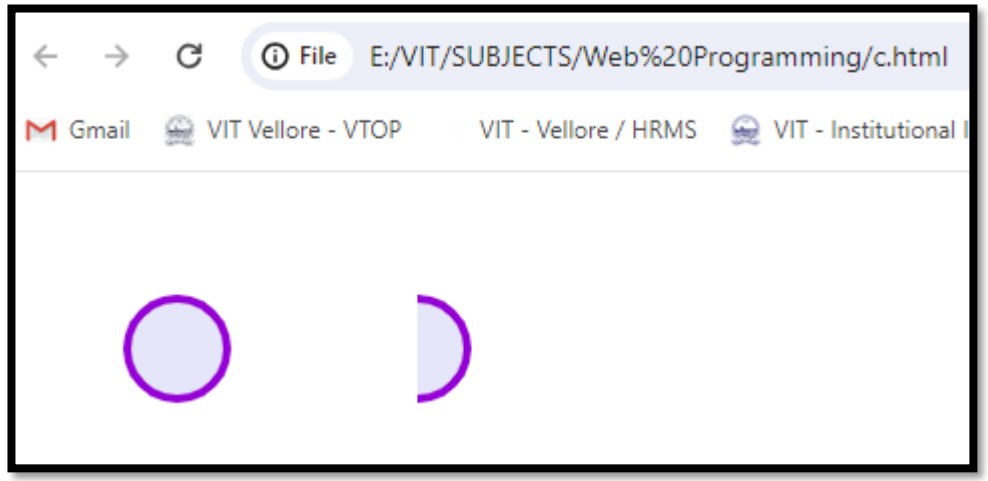


Example:

```
<!DOCTYPE html>
<head>
<script>
function displayQuotes() {
    var ctx; // the canvas object's context
    ctx = document.getElementById("canvas").getContext("2d");
    ctx.strokeStyle = "darkviolet";
    ctx.fillStyle = "lavender";
    ctx.lineWidth = 4;
    // Draw a circle
    ctx.beginPath();
    ctx.arc(80, 80, 25, 0, 2 * Math.PI);
    ctx.fill();
    ctx.stroke();
```

```
// Draw an arc in Counterclockwise direction
    ctx.beginPath();
    ctx.arc(200, 80, 25, .5 * Math.PI, 1.5 * Math.PI, true);
    ctx.fill();
    ctx.stroke();
} // end displayMsg
</script>
</head>
<body onload="displayQuotes();">
<canvas id="canvas" width="600" height="200">
</canvas>
</body>
</html>
```

False→ clockwise direction (default)
True→Counterclockwise



Drawing Lines and Paths

- *moveTo method(it moves the drawing pen)*

```
context.moveTo(starting-x, starting-y);
```

- *lineTo method(it moves the pen to the line's ending position)*

```
context.lineTo(ending-x, ending-y);
```

- *beginPath()(creating a path)*

```
context.beginPath();
```

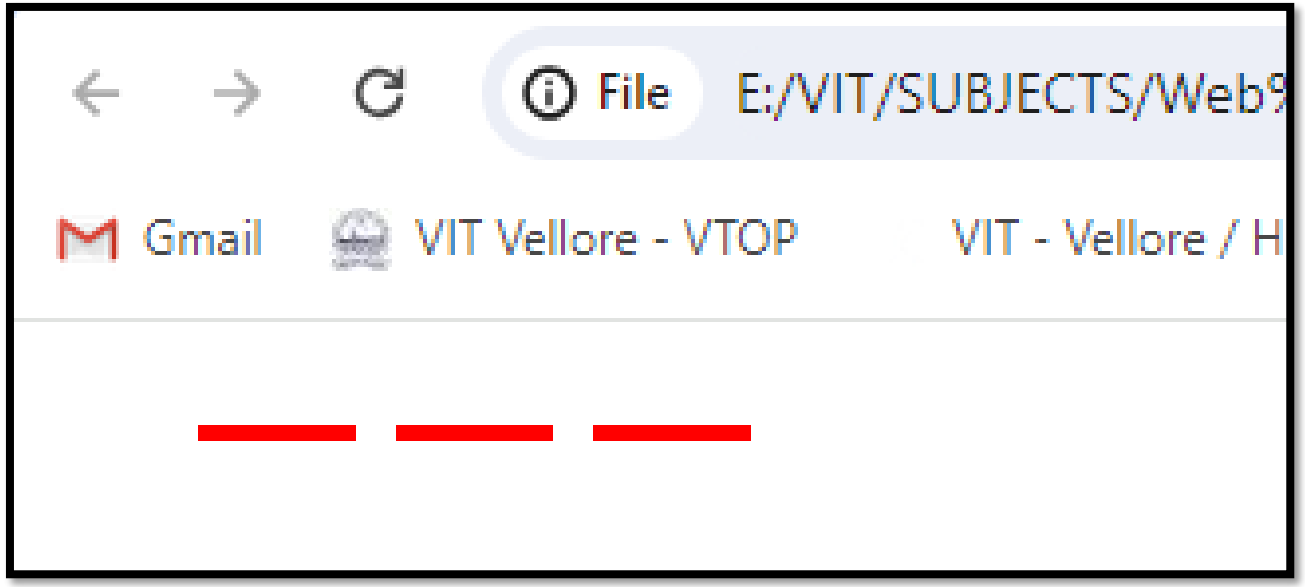
- *stroke(draws the current path)*

```
context.stroke();
```

Example:

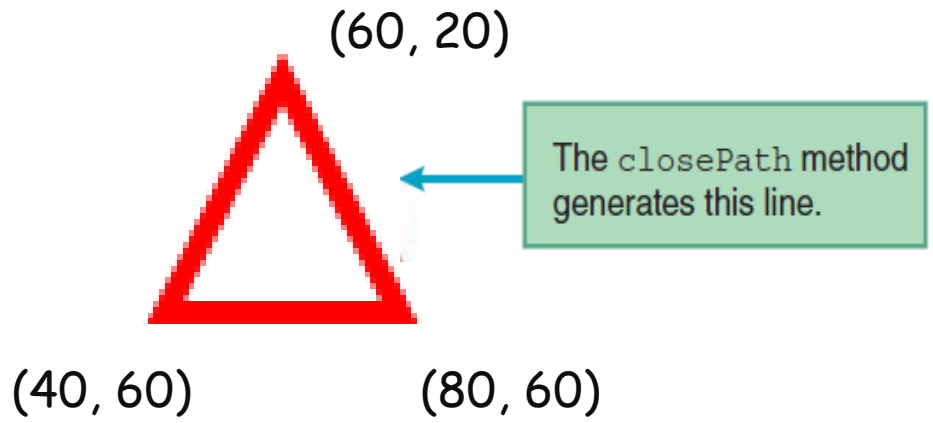
```
<!DOCTYPE html><head><script>
function displayQuotes() {
    var ctx; // the canvas object's context
    ctx = document.getElementById("canvas").getContext("2d");
    ctx.strokeStyle = "red";
    ctx.lineWidth = 4;
    ctx.beginPath();
    ctx.moveTo(40, 20);
    ctx.lineTo(80, 20);
    ctx.moveTo(90, 20);
    ctx.lineTo(130, 20);
    ctx.moveTo(140, 20);
    ctx.lineTo(180, 20);
    ctx.stroke();
}</script>
```

```
</head>
<body onload="displayQuotes();">
<canvas id="canvas" width="600" height="200">
</canvas>
</body>
</html>
```



Closing a Path

- *closePath method connects the most recently created line's end point to the most recently created subpath's starting point.*
  - ctx.moveTo(60, 20);
  - ctx.lineTo(40, 60);
  - ctx.lineTo(80, 60);
  - ctx.closePath();

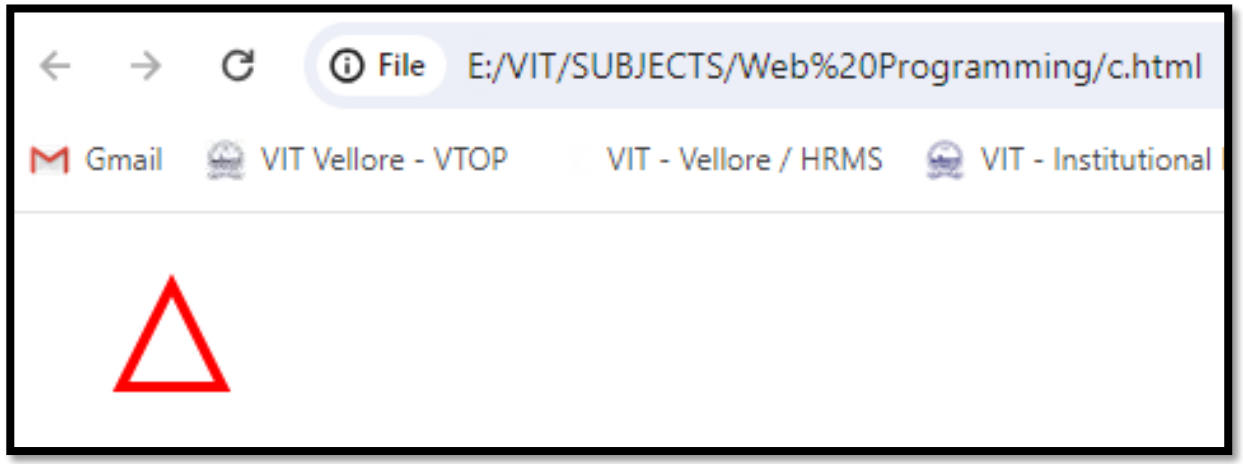


Connects the last point (80, 60) back to the starting point (60, 20), creating a closed shape. In this case, it forms a triangle with vertices at (60, 20), (40, 60), and (80, 60).

Example:

```
<!DOCTYPE html><head>
<script>
function displayQuotes() {
    var ctx; // the canvas object's context
    ctx = document.getElementById("canvas").getContext("2d");
   ctx.strokeStyle = "red";
ctx.lineWidth = 4;
ctx.beginPath();
ctx.moveTo(60, 20);
ctx.lineTo(40, 60);
ctx.lineTo(80, 60);
ctx.closePath();
ctx.stroke();
}</script>
```

```
</head>
<body onload="displayQuotes();">
<canvas id="canvas" width="600" height="200">
</canvas>
</body>
</html>
```

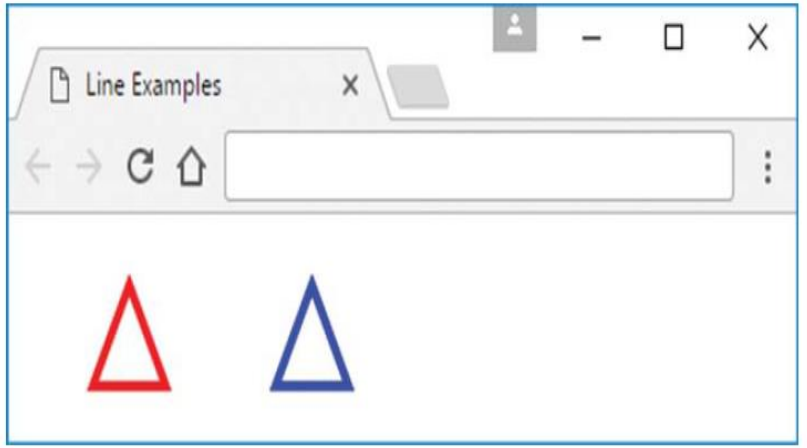


Splitting a Path

- if you want to change the line properties for a drawing, you need to split the path into multiple paths.
- And for each of the resulting smaller paths, you call beginPath at the start of the path and call stroke at the end of the path.

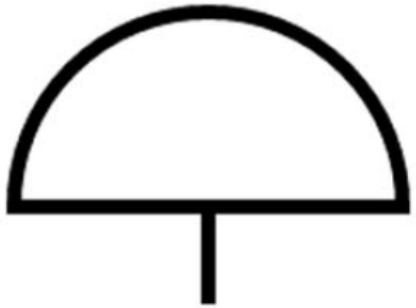
```
ctx.strokeStyle = "red";
ctx.lineWidth = 4;
ctx.beginPath();
ctx.moveTo(60, 20);
ctx.lineTo(40, 60);
ctx.lineTo(80, 60);
ctx.closePath();
ctx.stroke();

ctx.strokeStyle = "blue";
ctx.beginPath();
ctx.moveTo(160, 20);
ctx.lineTo(140, 60);
ctx.lineTo(180, 60);
ctx.closePath();
ctx.stroke();
```

Split the path so the color can change.

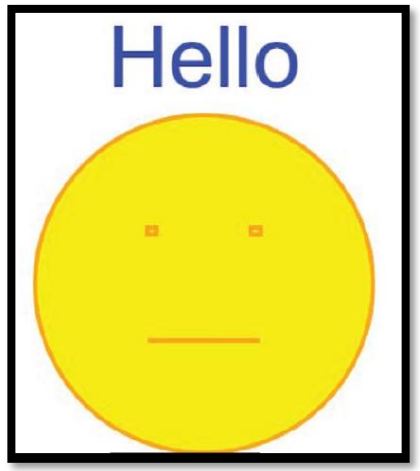


Practice Problems

- Simple Umbrella web page creation



- Face web page Creation



Using Canvas for Transformations

Translation:

```
context.translate(x, y);
```

The translate method moves the origin and the coordinate system *x* pixels to the right and *y* pixels down

Rotate:

```
context.rotate(clockwise-rotation-in-radians);
```

Rotation always takes place around the coordinate system's origin.

Scaling

```
context.scale(x-scale-factor, y-scale-factor);
```

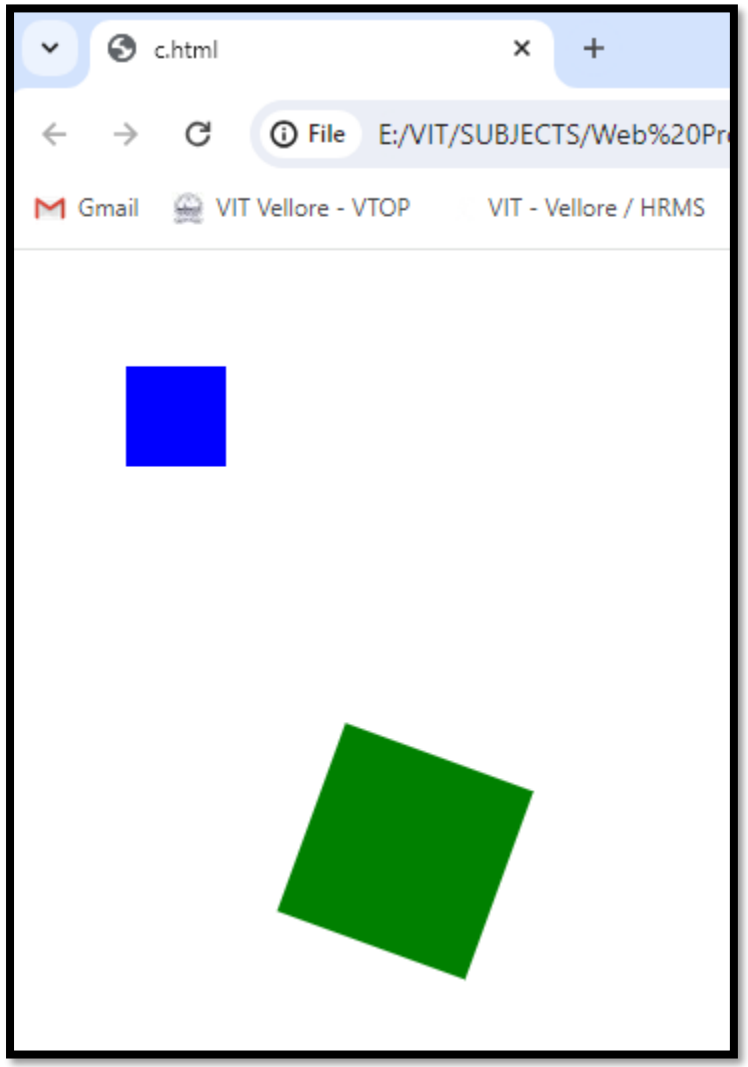
This method scales the current context..

Example:

```
<!DOCTYPE html><head>
<script>
function displayQuotes() {
    var ctx; // the canvas object's context
    ctx = document.getElementById("canvas").getContext("2d");
   // Draw a square before transformations
    ctx.fillStyle = "blue";
    ctx.fillRect(50, 50, 50, 50);
    // Translation, rotation, scale
    ctx.translate(100, 100);
    ctx.rotate(20 * Math.PI / 180);
    ctx.scale(2, 2);
    ctx.fillStyle = "green";
    ctx.fillRect(50, 50, 50, 50);
 }</script>
```

The Math.PI /180 part is where you do the converting to radians. You then multiply by 20 degrees you actually want.

```
</head>
<body onload="displayQuotes();">
<canvas id="canvas" width="600" height="400">
</canvas>
</body>
</html>
```



transform()

- This method scales, rotates, moves, and skews the context.
- ctx.transform(1, 0.5, -0.5, 1, 30, 10);

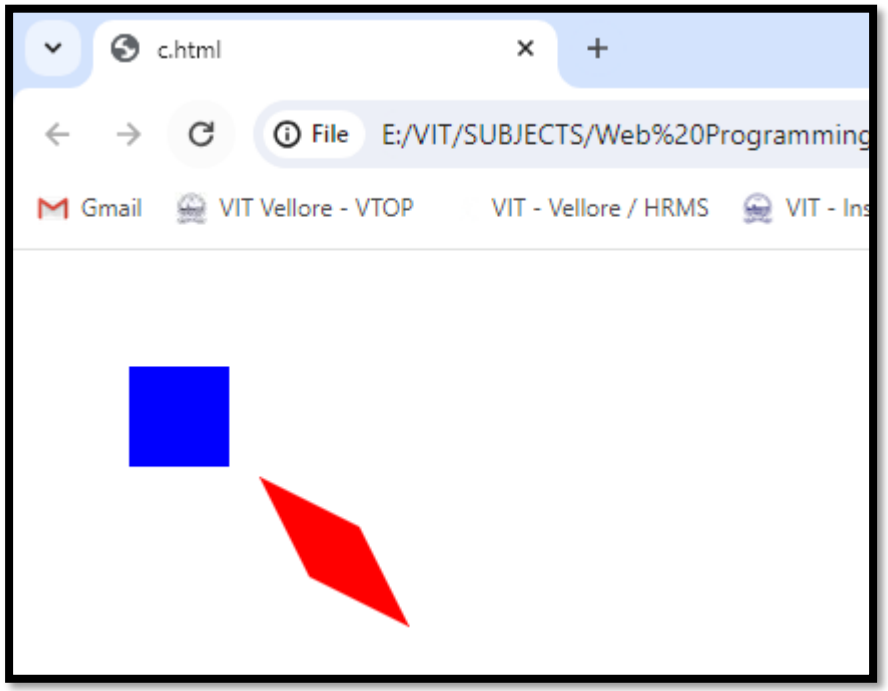
1. **Horizontal Scaling (m11): 1**
   - This parameter sets the horizontal scaling factor to **1**, meaning no horizontal scaling.
2. **Horizontal Skewing (m21): 0.5**
   - This parameter represents horizontal skewing. It sets the horizontal skewing factor to **0.5**, which skews the content horizontally.
3. **Vertical Skewing (m12): -0.5**
   - This parameter represents vertical skewing. It sets the vertical skewing factor to **-0.5**, which skews the content vertically.
4. **Vertical Scaling (m22): 1**
   - This parameter sets the vertical scaling factor to **1**, meaning no vertical scaling.
5. **Horizontal Translation (dx): 30**
   - This parameter translates the drawing horizontally by **30** units.
6. **Vertical Translation (dy): 10**
   - This parameter translates the drawing vertically by **10** units.

Example:

```
<!DOCTYPE html><head>
<script>
function displayQuotes() {
    var ctx; // the canvas object's context
    ctx = document.getElementById("canvas").getContext("2d");
    // Draw a square before transformation
      ctx.fillStyle = 'blue';
      ctx.fillRect(50, 50, 50, 50);
      // Apply a 2D transformation
      ctx.transform(1, 0.5, 0.5, 1, 30, 10);
      // Draw a square after transformation
      ctx.fillStyle = 'red';
      ctx.fillRect(50, 70, 50, 50);
  }
</script>
```

```
</head>
<body onload="displayQuotes();">
<canvas id="canvas" width="600" height="400">
</canvas>
</body>
</html>
```

It modifies the existing transformation matrix by multiplying it with the new matrix specified by the parameters.



setTransform()

- This method is used to set a new transformation matrix that includes horizontal scaling, horizontal skewing, vertical skewing, vertical scaling, horizontal translation, and vertical translation.

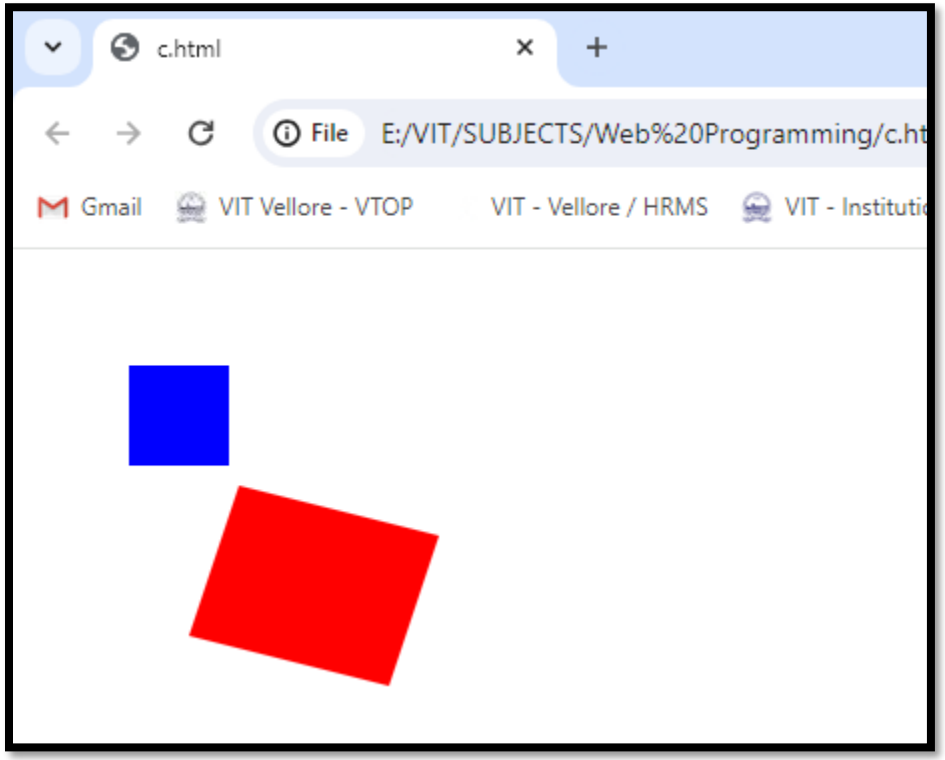
```
ctx.setTransform(2, 0.5, -0.5, 1.5, 30, 10)
```

- **a** (m11): Horizontal scaling by a factor of **2**.
- **b** (m21): Horizontal skewing by **0.5**.
- **c** (m12): Vertical skewing by **-0.5**.
- **d** (m22): Vertical scaling by a factor of **1.5**.
- **e** (dx): Horizontal translation by **30** units.
- **f** (dy): Vertical translation by **10** units.

Example:

```
<!DOCTYPE html><head>
<script>
function displayQuotes() {
    var ctx; // the canvas object's context
    ctx = document.getElementById("canvas").getContext("2d");
   // Draw a square before transformations
      ctx.fillStyle = 'blue';
      ctx.fillRect(50, 50, 50, 50);
      // Apply a set of transformations using setTransform
      ctx.setTransform(2, 0.5, -0.5, 1.5, 30, 10);
      // Draw a square after transformations
      ctx.fillStyle = 'red';
      ctx.fillRect(50, 50, 50, 50);
    }
</script>
```

```
</head>
<body onload="displayQuotes();">
<canvas id="canvas" width="600" height="400">
</canvas>
</body>
</html>
```



Regular Expression

- **A regular expression** is a sequence of characters that forms a **search pattern.**
- **Regular expressions** can be used to perform all types of text search and text replacement operations
- A regular expression can be a single character or a more complicated pattern.

Regular Expression Modifiers

Modifier	Description
i	Perform case-insensitive matching
g	Perform a global match (find all)
m	Perform multiline matching

Regular Expression Patterns

Expression	Description
[abc]	Find any character between the brackets
[^abc]	Find any character NOT between the brackets
[0-9]	Find any character between the brackets (any digit)
[^0-9]	Find any character NOT between the brackets (any non-digit)
(x)	A grouping or subpattern, which is also stored for later use
(x|y)	Find any of the alternatives specified

Metacharacters

Metacharacter	Description
.	Find a single character, except newline or line terminator
\w	Find a word character
\W	Find a non-word character
\d	Find a digit
\D	Find a non-digit character
\s	Find a whitespace character
\S	Find a non-whitespace character
\b	Find a match at the beginning/end of a word, beginning like this: \bHI, end like this: HI\b

Metacharacters

\B	Find a match, but not at the beginning/end of a word
\0	Find a NULL character
\n	Find a new line character
\f	Find a form feed character
\r	Find a carriage return character
\t	Find a tab character
\v	Find a vertical tab character
\xxx	Find the character specified by an octal number xxx
\xdd	Find the character specified by a hexadecimal number dd
\udddd	Find the Unicode character specified by a hexadecimal number dddd

Quantifiers

Quantifier	Description
*	Match zero or more times.
+	Match one or more times.
?	Match zero or one time.
{ n }	Match exactly *n* times.
{ n ,}	Match at least *n* times.
{ n , m }	Match from *n* to *m* times.

Test() method

Definition:

- The **test()** method is a regular expression method in JavaScript that checks whether a given string matches a specified pattern.

- **Syntax:**
  - **regex.test(str)**
- **Return Value:**
  - It returns a boolean value (**true** or **false**).
  - **true** indicates that the pattern was found in the string, while **false** indicates no match.

match() method

- **Definition:**
  - The **match()** method is another regular expression method in JavaScript used to retrieve the result of matching a string against a specified pattern.
- **Syntax:**
  - **str.match(regex)**
- **Return Value:**
  - It returns an array if a match is found, or **null** if no match is found.
  - The array contains the matched string and, if the regular expression contains capturing groups, additional elements for each captured group.

Example:

```
<!DOCTYPE html>
<head>      <title>Form Validation</title>
   <style type="text/css">
            .error {
          color: Red;
          visibility: hidden;
       }   </style>
</head>
<body>
   <h2>Form Validation</h2>
   <!-- PAN Card Validation -->
   <label for="txtPANCard">PAN Card:</label>
   <input name="txtPANCard" type="text" id="txtPANCard" class="PAN" />
   <span id="lblPANCard" class="error">Invalid PAN Number</span><br>  <br>
```

```
<!-- Aadhar Card Validation -->
  <label for="txtAadhar">Aadhar Card:</label>
  <input name="txtAadhar" type="text" id="txtAadhar" class="Aadhar" />
  <span id="lblAadhar" class="error">Invalid Aadhar Number</span>
 <br> <br>
  <!-- Phone Number Validation -->
  <label for="txtPhoneNumber">Phone Number:</label>
  <input name="txtPhoneNumber" type="text" id="txtPhoneNumber" class="PhoneNumber" />
  <span id="lblPhoneNumber" class="error">Invalid Phone Number</span>
  <br> <br>
  <!-- Email Validation -->
  <label for="txtEmail">Email:</label>
  <input name="txtEmail" type="text" id="txtEmail" class="Email" />
  <span id="lblEmail" class="error">Invalid Email</span>
 <br> <br>
  <input type="button" id="btnSubmit" value="Submit" onclick="validateForm()" />
```

```
<script type="text/javascript">
        function validateForm() {
            // PAN Card Validation
            var txtPANCard = document.getElementById("txtPANCard");
            var lblPANCard = document.getElementById("lblPANCard");
            var panRegex = /([A-Z]){5}([0-9]){4}([A-Z]){1}$/;

            // Aadhar Card Validation
            var txtAadhar = document.getElementById("txtAadhar");
            var lblAadhar = document.getElementById("lblAadhar");
            var aadharRegex = /^\d{12}$/;
```

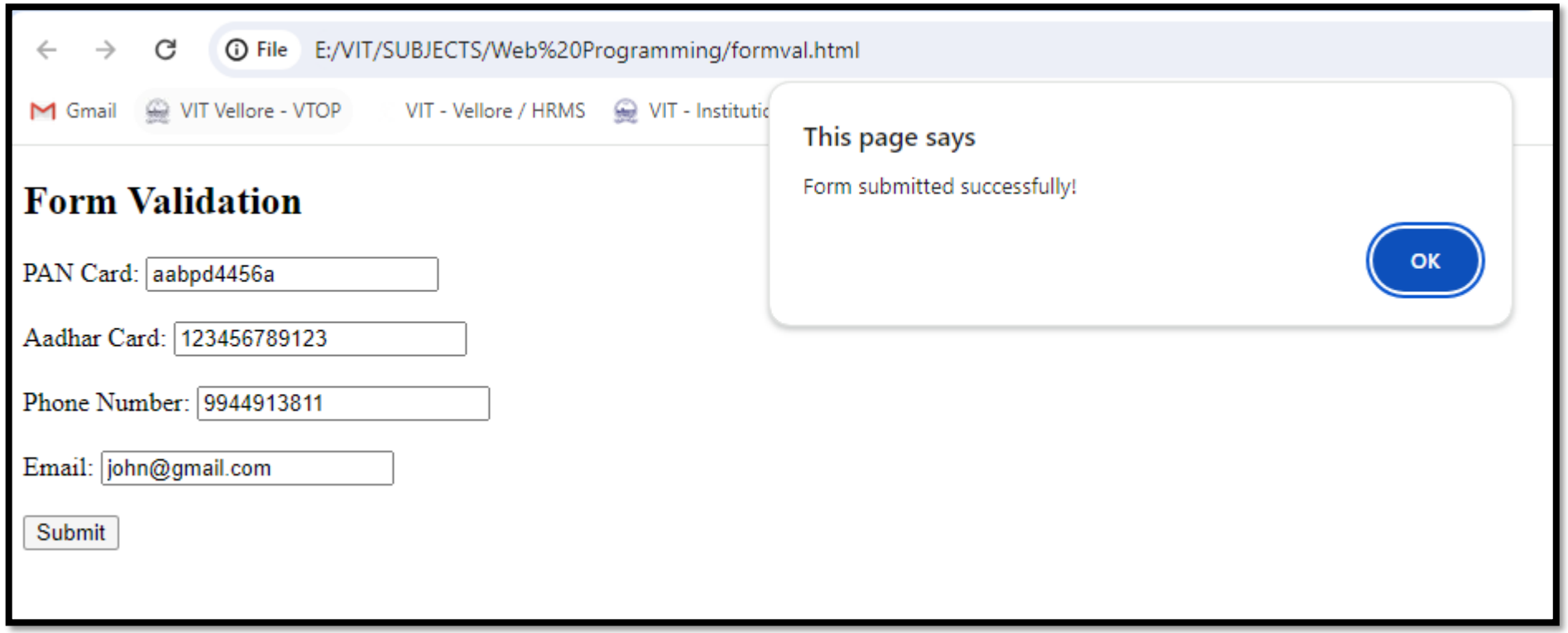
```
// Phone Number Validation
        var txtPhoneNumber =
document.getElementById("txtPhoneNumber");
        var lblPhoneNumber =
document.getElementById("lblPhoneNumber");
        var phoneRegex = /^[789]\d{9}$/;
        // Email Validation
        var txtEmail = document.getElementById("txtEmail");
        var lblEmail = document.getElementById("lblEmail");
        var emailRegex = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
```

```
// Validate PAN
        if (panRegex.test(txtPANCard.value.toUpperCase())) {
            lblPANCard.style.visibility = "hidden";
        } else {
            lblPANCard.style.visibility = "visible";
        }
    // Validate Aadhar
        if (aadharRegex.test(txtAadhar.value)) {
            lblAadhar.style.visibility = "hidden";
        } else {
            lblAadhar.style.visibility = "visible";
        }
```

```
// Validate Phone Number
        if (phoneRegex.test(txtPhoneNumber.value)) {
            lblPhoneNumber.style.visibility = "hidden";
        } else {
            lblPhoneNumber.style.visibility = "visible";
        }
        // Validate Email
        if (emailRegex.test(txtEmail.value)) {
            lblEmail.style.visibility = "hidden";
        } else {
            lblEmail.style.visibility = "visible";
        }
```

```
// Check if all validations passed
            if (lblPANCard.style.visibility === "hidden" &&
                lblAadhar.style.visibility === "hidden" &&
                lblPhoneNumber.style.visibility === "hidden" &&
                lblEmail.style.visibility === "hidden") {
                // Display successful alert
                alert("Form submitted successfully!");
            }
        }
    </script>
</body>
</html>
```


Form Validation
PAN Card: aabpd4456
Invalid PAN Number
Aadhar Card: 12345678912
Invalid Aadhar Number
Phone Number: 994491381
Invalid Phone Number
Email: johngmail.com
Invalid Email
Submit


File
E:/VIT/SUBJECTS/Web%20Programming/formval.html
Gmail
VIT Vellore - VTOP
VIT - Vellore / HRMS
VIT - Institutic
This page says
Form submitted successfully!
OK
Form Validation
PAN Card: aabpd4456a
Aadhar Card: 123456789123
Phone Number: 9944913811
Email: john@gmail.com
Submit